UNIVERSIDADE TUIUTI DO PARANÁ

JOÃO THOMAZ VIEIRA
KLAUS SIEGFRIED BECKMANN

FERRAMENTA BASEADA EM LINGUAGEM DE DOMÍNIO ESPECÍFICO PARA O APOIO AO ENSINO DE AUTÔMATOS FINITOS COM SIMULAÇÃO VISUAL INTERATIVA

CURITIBA
2026

JOÃO THOMAZ VIEIRA
KLAUS SIEGFRIED BECKMANN

FERRAMENTA BASEADA EM LINGUAGEM DE DOMÍNIO ESPECÍFICO PARA O APOIO AO ENSINO DE AUTÔMATOS FINITOS COM SIMULAÇÃO VISUAL INTERATIVA

Trabalho de Conclusão de Curso apresentado ao curso de Bacharelado em Ciência da Computação da Faculdade de Ciências Exatas e de Tecnologia da Universidade Tuiuti do Paraná, como requisito à obtenção do grau de Bacharel.

Orientador: Prof. Diógenes Cogo Furlan

CURITIBA
2026

RESUMO

O ensino de autômatos finitos em cursos de Ciência da Computação enfrenta dificuldades relacionadas à abstração dos conceitos teóricos e à falta de retroalimentação imediata, uma vez que as ferramentas existentes adotam abordagens exclusivamente visuais, sem oferecer representação textual formal, versionamento ou diagnósticos de erro precisos. Este trabalho apresenta a Autosim, uma ferramenta educacional composta por uma Linguagem de Domínio Específico (DSL) textual, em português, para a descrição declarativa de Autômatos Finitos Determinísticos (AFD) e Não-Determinísticos (AFN) com transições ε, cuja gramática foi formalizada em EBNF. A ferramenta inclui um compilador, implementado em Rust, que realiza análise léxica, sintática e semântica e fornece mensagens de erro com indicação de linha, coluna e natureza do problema, além de uma interface gráfica interativa que renderiza automaticamente o diagrama de estados a partir do texto e anima a simulação passo a passo, destacando os estados e as transições percorridos durante o reconhecimento de cadeias de entrada. A validação foi realizada por meio de 51 testes automatizados, cobrindo as fases de análise e o simulador, de simulações com autômatos clássicos da literatura e de cenários com erros propositais; todos os resultados concordaram com o esperado, confirmando a correção do simulador e a qualidade dos diagnósticos gerados. A ferramenta conecta, em um único instrumento, a teoria de autômatos à prática de compiladores, oferecendo ao estudante um ambiente para descrever, validar e executar autômatos finitos.

Palavras-chave: autômatos finitos; linguagem de domínio específico; compiladores; simulação visual; ensino de computação.

LISTA DE FIGURAS

FIGURA 1 – ÁRVORE DE DERIVAÇÃO DA PALAVRA $aabb$ NA GRAMÁTICA G 17
FIGURA 2 – AFD QUE RECONHECE CADEIAS CONTENDO 01 COMO SUBCADEIA 20
FIGURA 3 – AFN QUE RECONHECE CADEIAS TERMINADAS EM 01 22
FIGURA 4 – AFD CONSTRUÍDO A PARTIR DO AFN DA FIGURA 3 PELA CONSTRUÇÃO DE SUBCONJUNTOS 25
FIGURA 5 – FASES DE UM COMPILADOR 34
FIGURA 6 – INTERAÇÕES ENTRE O ANALISADOR LÉXICO E O ANALISADOR SINTÁTICO 36
FIGURA 7 – ÁRVORE DE DERIVAÇÃO DA EXPRESSÃO $id + id * id$ 39
FIGURA 8 – ÁRVORE SINTÁTICA ABSTRATA DA EXPRESSÃO $id + id * id$ . . 43
FIGURA 9 – GRAMÁTICA DA DSL AUTOSIM EM NOTAÇÃO EBNF 69
FIGURA 10 – INTERFACE DA FERRAMENTA AUTOSIM RENDERIZANDO O AFN-ε DA LISTAGEM 6.2 72
FIGURA 11 – PIPELINE DE COMPILAÇÃO DA FERRAMENTA AUTOSIM . . . 74
FIGURA 12 – MENSAGEM DE ERRO LÉXICO GERADA PELA BIBLIOTECA ARIADNE 75
FIGURA 13 – VISÃO GERAL DA INTERFACE DA FERRAMENTA AUTOSIM . . 82
FIGURA 14 – DIAGRAMA DE UM AFD DE 14 ESTADOS EXCEDENDO OS LIMITES DA JANELA 84
FIGURA 15 – SIMULAÇÃO CONCLUÍDA COM VEREDITO DE ACEITAÇÃO . . 85
FIGURA 16 – SIMULAÇÃO CONCLUÍDA COM VEREDITO DE REJEIÇÃO . . . 86

LISTA DE QUADROS

QUADRO 1 – HIERARQUIA DE CHOMSKY 18
QUADRO 2 – TABELA DE TRANSIÇÃO DO AFD QUE RECONHECE CADEIAS CONTENDO 01 26
QUADRO 3 – COMPARAÇÃO ENTRE BIBLIOTECAS DE PARSER COMBINATORS PARA RUST 58
QUADRO 4 – COMPARAÇÃO DE RECURSOS TÉCNICOS ENTRE OS TRABALHOS RELACIONADOS 65
QUADRO 5 – CONTRIBUIÇÕES E LACUNAS IDENTIFICADAS NA LITERATURA 66
QUADRO 6 – TERMINAIS DA DSL AUTOSIM 69
QUADRO 7 – PALAVRAS RESERVADAS DA DSL AUTOSIM 70
QUADRO 8 – RESUMO DOS TESTES AUTOMATIZADOS 87
QUADRO 9 – SIMULAÇÕES REALIZADAS COM AUTÔMATOS CLÁSSICOS . 93
QUADRO 10 – CENÁRIOS DE TESTE DE DIAGNÓSTICO DE ERRO 93

LISTA DE SIGLAS E ACRÔNIMOS

AFD	Autômato Finito Determinístico
AFN	Autômato Finito Não-Determinístico
AST	*Abstract Syntax Tree* (Árvore Sintática Abstrata)
BNF	*Backus-Naur Form* (Forma de Backus-Naur)
DSL	*Domain-Specific Language* (Linguagem de Domínio Específico)
EBNF	*Extended Backus-Naur Form* (Forma de Backus-Naur Estendida)
FSM	*Finite State Machine* (Máquina de Estados Finitos)
GNU	*GNU's Not Unix* (GNU Não é Unix)
GTK	*GIMP Toolkit* (Conjunto de Ferramentas do GIMP)
JVM	*Java Virtual Machine* (Máquina Virtual Java)
LALR	*Look-Ahead LR* (LR com Visualização de Próximo Símbolo)
LL	*Left-to-right, Leftmost derivation* (Esquerda para a direita, Derivação mais à esquerda)
LLVM	*Low Level Virtual Machine* (Máquina Virtual de Baixo Nível)
LR	*Left-to-right, Rightmost derivation* (Esquerda para a direita, Derivação mais à direita)
PEG	*Parsing Expression Grammar* (Gramática de Expressão de Análise)
SQL	*Structured Query Language* (Linguagem de Consulta Estruturada)
UFSC	Universidade Federal de Santa Catarina
URL	*Uniform Resource Locator* (Localizador Uniforme de Recursos)

LISTA DE SÍMBOLOS

Σ	Alfabeto
Σ^*	Conjunto de todas as palavras sobre Σ
ε	Palavra vazia
$\|w\|$	Comprimento da palavra w
$\varnothing$	Conjunto vazio
$\in$	Pertence a
$\notin$	Não pertence a
$\subseteq$	Subconjunto
$\cup$	União
$\cap$	Interseção
$\mathcal{P}(Q)$	Conjunto das partes de Q
L^*	Fecho de Kleene da linguagem L
$\rightarrow$	Regra de produção
$\Rightarrow$	Derivação direta
$\Rightarrow^*$	Derivação em zero ou mais passos
δ	Função de transição
$\hat{\delta}$	Função de transição estendida a cadeias
$E(q)$	ε-fecho do estado q
$L(M)$	Linguagem reconhecida pelo autômato M
$L(G)$	Linguagem gerada pela gramática G

SUMÁRIO

1 INTRODUÇÃO 10

FUNDAMENTAÇÃO TEÓRICA 13

2 AUTÔMATOS FINITOS E LINGUAGENS FORMAIS 14

2.1 CONCEITOS FUNDAMENTAIS DE LINGUAGENS FORMAIS 14

2.2 GRAMÁTICAS FORMAIS 16

2.3 AUTÔMATO FINITO DETERMINÍSTICO 18

2.4 AUTÔMATO FINITO NÃO-DETERMINÍSTICO 20

2.5 EQUIVALÊNCIA ENTRE AFD E AFN 23

2.6 REPRESENTAÇÕES DE AUTÔMATOS 25

2.7 SIMULAÇÃO DE AUTÔMATOS 27

2.8 APLICAÇÕES DE AUTÔMATOS 30

3 COMPILADORES E LINGUAGENS DE DOMÍNIO ESPECÍFICO 32

3.1 CONCEITOS FUNDAMENTAIS DE COMPILADORES 32

3.2 ANÁLISE LÉXICA 35

3.3 ANÁLISE SINTÁTICA 38

3.4 ANÁLISE SEMÂNTICA 40

3.5 AST (ÁRVORE SINTÁTICA ABSTRATA) 42

3.6 LINGUAGENS DE DOMÍNIO ESPECÍFICO (DSL) 44

3.7 CONSTRUÇÃO DE COMPILADORES (FERRAMENTAS) 47

3.8 DIAGNÓSTICO DE ERROS 49

3.9 DSL NO ENSINO 52

4 RUST COMO LINGUAGEM DE IMPLEMENTAÇÃO 54

4.1 VISÃO GERAL DA LINGUAGEM 54

4.2 SISTEMA DE TIPOS, *OWNERSHIP* E *BORROWING* 55

4.3 ECOSSISTEMA E GERENCIADOR DE PACOTES CARGO 56

4.4 *PARSER COMBINATORS* EM RUST 57

5 TRABALHOS RELACIONADOS 60

5.1 ANÁLISE LÉXICA EM RUST 60

5.2 JFLAP 61

5.3 GAM 62
5.4 SIMULADOR UFSC 62
5.5 AUTOMATOGRAPH 63
5.6 ANÁLISE COMPARATIVA 64
6 PROJETO DA DSL 67
6.1 VISÃO GERAL DA LINGUAGEM 67
6.2 GRAMÁTICA FORMAL 68
6.3 PALAVRAS RESERVADAS 69
6.4 DECISÕES DE PROJETO 70
6.4.1 Declaração Única de Alfabeto 70
6.4.2 Classificação Explícita AFD/AFN 70
6.4.3 Sintaxe em Português 71
6.4.4 Insensibilidade a Maiúsculas e Minúsculas 71
6.5 SUPORTE A TRANSIÇÕES ε 71
6.6 TRATAMENTO DE COMENTÁRIOS 73
7 IMPLEMENTAÇÃO 74
7.1 VISÃO GERAL DO PIPELINE 74
7.2 ANALISADOR LÉXICO 74
7.2.1 Definição dos Tokens 74
7.2.2 Tratamento de Erros Léxicos 75
7.3 ANALISADOR SINTÁTICO 76
7.3.1 Árvore Sintática Abstrata (AST) 76
7.3.2 Estrutura da Gramática 77
7.3.3 Tratamento de Erros Sintáticos 77
7.4 ANALISADOR SEMÂNTICO 77
7.4.1 Construção da Tabela de Símbolos 77
7.4.2 Verificação de Referências 78
7.4.3 Verificação de Determinismo 78
7.5 SIMULADOR 79
7.5.1 Estratégia de Exibição 79
7.5.2 Seleção do Caminho 79
7.5.3 Passo de Simulação 80

7.6 INTERFACE GRÁFICA 81
7.6.1 Layout da Interface 81
7.6.2 Renderização do Diagrama de Estados 82
7.6.3 Simulação Visual Passo a Passo 84
7.7 DIAGNÓSTICO DE ERROS COM ARIADNE 86
8 RESULTADOS 87
8.1 ESTRATÉGIA DE TESTES 87
8.2 TESTES DO ANALISADOR LÉXICO 87
8.3 TESTES DO ANALISADOR SINTÁTICO 88
8.4 TESTES DO ANALISADOR SEMÂNTICO 90
8.5 TESTES DO SIMULADOR 91
8.6 SIMULAÇÕES COM AUTÔMATOS CLÁSSICOS 92
8.7 CENÁRIOS DE DIAGNÓSTICO DE ERRO 92
9 CONCLUSÃO 94
REFERÊNCIAS 96

1 INTRODUÇÃO

O estudo de linguagens formais e de autômatos finitos é um dos pilares da formação em Ciência da Computação, estando diretamente relacionado à compreensão das máquinas abstratas, dos limites da computação e da teoria por trás dos compiladores. Apesar de sua importância, essa disciplina é reconhecidamente desafiadora para muitos estudantes que, conforme relatam Morazán e Antunez (2014), chegam a percebê-la como excessivamente difícil e distante das demais atividades do curso. Os autores atribuem essa percepção a uma causa precisa: ao contrário do que ocorre em disciplinas de programação, o estudante é solicitado a projetar autômatos e gramáticas, além de provar sua correção, sem qualquer possibilidade de experimentar sua construção ou de obter retroalimentação imediata sobre ela. Esse distanciamento entre o formalismo simbólico e a prática computacional evidencia a necessidade de ferramentas didáticas que permitam descrever, validar e executar tais construções de forma interativa.

Nesse contexto, ferramentas disponíveis atualmente para apoiar esse aprendizado, como o JFLAP (Rodger; Finley, 2006), adotam uma abordagem exclusivamente visual baseada em arrastar e soltar, na qual o usuário constrói o autômato posicionando estados e conectando transições com o *mouse*. Embora essa abordagem seja útil para ilustrar conceitos, ela apresenta limitações relevantes: a descrição do autômato não é textual, o que dificulta o versionamento e o compartilhamento dos modelos; não há validação formal da estrutura com mensagens de erro claras e localizadas; e o estudante que comete um erro na construção não recebe um diagnóstico preciso do problema.

Diante desse cenário, este trabalho propõe o desenvolvimento de uma ferramenta educacional que adota uma abordagem baseada em texto formal e compilação pedagógica. A principal contribuição desta solução é o acoplamento direto entre uma *Domain-Specific Language* (DSL), ou seja, uma linguagem de domínio específico textual, e a geração automática de uma interface gráfica interativa sincronizada. Isso diferencia a ferramenta do JFLAP de duas formas fundamentais: primeiro, substitui a modelagem puramente visual e propensa a erros de clique por uma descrição declarativa estruturada, permitindo o uso de ferramentas de versionamento de código

(como o Git); segundo, introduz uma infraestrutura de compilador real (com análises léxica, sintática e semântica) focada no diagnóstico detalhado de erros conceituais (como transições para estados não declarados ou determinismo violado), fornecendo mensagens com linha e coluna exatas.

Do ponto de vista pedagógico, a ferramenta busca mitigar o problema específico da ausência de retroalimentação imediata que ocorre na transição entre o formalismo matemático e a prática computacional. Em vez de testar sua máquina por tentativa e erro visual, o estudante passa a interagir com os diagnósticos formais de um compilador, o que aproxima a teoria de autômatos da prática de compiladores em um único instrumento educacional. A solução unificada busca, assim, guiar o aprendizado por meio do rigor linguístico e da depuração contextual.

O objetivo geral deste trabalho é desenvolver essa ferramenta educacional, que integra a descrição textual de autômatos, sua compilação pedagógica e sua simulação visual em um único ambiente. Para alcançá-lo, foram estabelecidos os seguintes objetivos específicos:

a) projetar uma DSL textual com sintaxe declarativa própria para a descrição formal de AFDs e AFNs, com a gramática formalizada na notação *Extended Backus-Naur Form* (EBNF), ou seja, a forma de Backus-Naur estendida;

b) implementar um compilador pedagógico para a DSL com análise léxica, sintática e semântica, focado na detecção precoce de erros de consistência formal e na geração de mensagens de diagnóstico localizadas;

c) desenvolver um simulador capaz de processar cadeias de entrada sobre os autômatos descritos na DSL, determinando aceitação ou rejeição e registrando detalhadamente as configurações intermediárias;

d) construir uma interface gráfica interativa que renderize automaticamente o diagrama de estados a partir do texto fonte e anime a simulação passo a passo, permitindo o rastreamento síncrono da execução;

e) validar a correção do simulador e a capacidade diagnóstica do compilador por meio de testes automatizados, de simulações com autômatos clássicos da literatura e de cenários com falhas estruturais induzidas.

Este trabalho está organizado como segue. O Capítulo 2 apresenta os conceitos

teóricos de autômatos finitos e linguagens formais. O Capítulo 3 aborda linguagens de domínio específico e as fases de compilação. O Capítulo 4 apresenta a linguagem Rust e justifica sua adoção como plataforma de implementação da ferramenta proposta. O Capítulo 5 discute as ferramentas relacionadas e suas limitações. O Capítulo 6 descreve o projeto da DSL, sua sintaxe e gramática. O Capítulo 7 detalha a implementação do compilador e da interface gráfica. O Capítulo 8 apresenta os testes e os resultados obtidos. Por fim, o Capítulo 9 apresenta as considerações finais e trabalhos futuros.

FUNDAMENTAÇÃO TEÓRICA

2 AUTÔMATOS FINITOS E LINGUAGENS FORMAIS

2.1 CONCEITOS FUNDAMENTAIS DE LINGUAGENS FORMAIS

O estudo dos autômatos finitos exige a compreensão prévia de algumas estruturas matemáticas elementares: conjuntos, alfabetos, palavras e linguagens. Esta seção apresenta tais conceitos de forma sistemática, fixando a notação e a terminologia adotadas nos capítulos subsequentes.

Um conjunto é uma coleção de elementos, que podem ser de qualquer natureza, como números, letras, símbolos ou mesmo outros conjuntos. Para indicar que um elemento a pertence a um conjunto C, escreve-se $a \in C$. O símbolo $\notin$ denota a relação inversa. Por exemplo, dado $C = \{a, b, c\}$, tem-se $a \in C$ e $d \notin C$. Conjuntos podem ser finitos ou infinitos. No segundo caso, é usual o emprego de reticências para indicar a continuação dos elementos, como em $\mathbb{Z} = \{\ldots, -2, -1, 0, 1, 2, \ldots\}$. O conjunto que não contém qualquer elemento é denominado conjunto vazio e é representado pelo símbolo $\varnothing$ (Sipser, 2012).

A partir desta noção de conjunto, define-se a primeira estrutura específica da teoria das linguagens formais: o alfabeto. Um alfabeto é um conjunto finito e não vazio de símbolos, representado, por convenção, pela letra grega Σ (Sipser, 2012). Os elementos de um alfabeto são denominados símbolos ou letras. Por exemplo, $\Sigma_1 = \{0, 1\}$ é o alfabeto binário, enquanto $\Sigma_2 = \{a, b, c, \ldots, z\}$ corresponde ao alfabeto latino minúsculo. A exigência de finitude é fundamental, pois garante que os autômatos, definidos nas seções seguintes, possam manipular o alfabeto de forma efetiva. A exigência de que ele não seja vazio, por sua vez, assegura que sempre haja ao menos um símbolo disponível para a formação de palavras (Diverio; Menezes, 2011; Vieira, 2004).

Sobre um alfabeto Σ, define-se uma palavra (ou cadeia) como uma sequência finita de símbolos pertencentes a Σ, escritos de forma justaposta e sem separadores. O comprimento de uma palavra w, denotado por $|w|$, corresponde ao número de símbolos que a compõem. A palavra de comprimento zero, denominada palavra vazia, é representada pelo símbolo ε (Sipser, 2012). O conjunto de todas as palavras que podem ser formadas sobre Σ, incluindo a palavra vazia, é denotado por Σ^*. Assim, para $\Sigma = \{a, b\}$, tem-se $\Sigma^* = \{\varepsilon, a, b, aa, ab, ba, bb, aaa, \ldots\}$, conjunto que é sempre infinito e

enumerável (Vieira, 2004). Estabelecidas as noções de alfabeto e palavra, é possível definir o objeto central deste trabalho: a linguagem. Uma linguagem formal sobre um alfabeto Σ é qualquer subconjunto de Σ^*, ou seja, qualquer conjunto de palavras formadas a partir dos símbolos de Σ (Sipser, 2012; Vieira, 2004). Linguagens podem ser finitas, como $L_1 = \{a, ab, abb\}$, ou infinitas, como $L_2 = \{a^n b^n \mid n \geq 1\}$, que contém todas as palavras compostas por uma sequência de a's seguida de igual número de b's. A maior parte das linguagens de interesse teórico e prático é infinita, o que motiva o desenvolvimento de mecanismos formais, como gramáticas e autômatos, capazes de descrevê-las de maneira finita.

Por serem conjuntos, as linguagens herdam diretamente as operações da teoria de conjuntos. Dadas duas linguagens L_1 e L_2 sobre um mesmo alfabeto Σ, define-se a união $L_1 \cup L_2 = \{w \mid w \in L_1$ ou $w \in L_2\}$, o conjunto das palavras que pertencem a pelo menos uma das duas; a interseção $L_1 \cap L_2 = \{w \mid w \in L_1$ e $w \in L_2\}$, formada pelas palavras comuns a ambas; e a diferença $L_1 - L_2 = \{w \mid w \in L_1$ e $w \notin L_2\}$, composta pelas palavras de L_1 que não ocorrem em L_2 (Diverio; Menezes, 2011; Vieira, 2004). Por exemplo, para $\Sigma = \{a, b\}$, $L_1 = \{\varepsilon, a, ab\}$ e $L_2 = \{a, b\}$, tem-se $L_1 \cup L_2 = \{\varepsilon, a, b, ab\}$, $L_1 \cap L_2 = \{a\}$ e $L_1 - L_2 = \{\varepsilon, ab\}$.

Além dessas, duas operações específicas, derivadas da estrutura sequencial das palavras, desempenham papel central na teoria. A concatenação de duas palavras $x = a_1 a_2 \cdots a_m$ e $y = b_1 b_2 \cdots b_n$ produz a palavra $xy = a_1 a_2 \cdots a_m b_1 b_2 \cdots b_n$. Em particular, $w\varepsilon = \varepsilon w = w$ para qualquer palavra w, o que faz de ε o elemento neutro dessa operação. A concatenação estende-se naturalmente a linguagens: dadas L_1 e L_2, define-se $L_1 L_2 = \{xy \mid x \in L_1$ e $y \in L_2\}$. A partir dela, define-se a potência de uma linguagem por $L^0 = \{\varepsilon\}$ e $L^n = L^{n-1} L$ para $n \geq 1$. Por fim, o fecho de Kleene de uma linguagem L, denotado por L^*, corresponde ao conjunto de todas as palavras obtidas pela concatenação de zero ou mais elementos de L, isto é, $L^* = \bigcup_{n \in \mathbb{N}} L^n$ (Diverio; Menezes, 2011; Vieira, 2004). Essas operações constituem a base para a especificação concisa de linguagens, recurso amplamente explorado nas expressões regulares e gramáticas estudadas nas seções seguintes.

2.2 GRAMÁTICAS FORMAIS

Conforme observado na seção anterior, a maior parte das linguagens de interesse prático é infinita, o que torna inviável descrevê-las pela enumeração explícita de suas palavras. Faz-se necessário, portanto, um mecanismo finito capaz de gerar todas as palavras de uma linguagem. Entre os formalismos propostos para esse fim, as gramáticas ocupam posição de destaque, tanto pelo seu poder expressivo quanto pela estreita relação com os autômatos estudados nas seções subsequentes (Lewis; Papadimitriou, 1998; Vieira, 2004).

Uma gramática descreve uma linguagem por meio de um conjunto finito de regras de produção, que indicam como certas sequências de símbolos podem ser substituídas por outras. Nessa estrutura, distinguem-se dois tipos de símbolos: os terminais, que compõem o alfabeto da linguagem gerada e aparecem nas palavras finais, e os não-terminais (ou variáveis), símbolos auxiliares que servem de intermediários no processo de geração e não fazem parte das palavras produzidas (Lewis; Papadimitriou, 1998; Sipser, 2012). A geração de uma palavra inicia-se sempre a partir de uma variável especial, denominada símbolo inicial (ou variável de partida), e prossegue pela aplicação sucessiva das regras até que reste apenas uma sequência de terminais.

Formalmente, uma gramática é uma quádrupla $G = (V, \Sigma, R, S)$, na qual (Sipser, 2012; Diverio; Menezes, 2011; Vieira, 2004):

- V é um conjunto finito de variáveis (símbolos não-terminais);
- Σ é um alfabeto de símbolos terminais, com $V \cap \Sigma = \varnothing$;
- R é um conjunto finito de regras de produção, cada uma escrita na forma $\alpha \to \beta$, em que $\alpha, \beta \in (V \cup \Sigma)^*$ e o lado esquerdo α contém ao menos uma variável;
- $S \in V$ é a variável de partida.

A aplicação de uma regra $\alpha \to \beta \in R$ a uma cadeia de variáveis e terminais produz uma nova cadeia pela substituição de uma ocorrência de α por β. Essa relação de substituição direta é denotada por $\Rightarrow$: dadas $u, v \in (V \cup \Sigma)^*$, escreve-se $u \Rightarrow v$ quando existem $x, y \in (V \cup \Sigma)^*$ e uma regra $\alpha \to \beta \in R$ tais que $u = x\alpha y$ e $v = x\beta y$ (Lewis; Papadimitriou, 1998). Em outras palavras, x e y representam o trecho da cadeia que permanece inalterado, enquanto a subcadeia α, situada entre eles, é trocada por

β. A título de exemplo, dada a regra $A \to ab$ e a cadeia cAd, tem-se $cAd \Rightarrow cabd$, identificando-se $x = c$, $\alpha = A$, $y = d$ e $\beta = ab$.

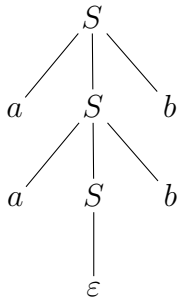
A aplicação sucessiva de regras, isto é, o fecho reflexivo e transitivo de $\Rightarrow$, é denotada por $\Rightarrow^*$. A linguagem gerada pela gramática G é o conjunto de todas as palavras formadas exclusivamente por terminais que podem ser obtidas a partir do símbolo inicial: $L(G) = \{w \in \Sigma^* \mid S \Rightarrow^* w\}$ (Sipser, 2012; Vieira, 2004).

A título de ilustração, considere a gramática $G = (\{S\}, \{a, b\}, R, S)$, em que $R = \{S \to aSb,\ S \to \varepsilon\}$. Partindo do símbolo inicial S, é possível obter, por exemplo, a palavra $aabb$ por meio da seguinte derivação:

$$
\begin{aligned}
S &\Rightarrow aSb && \text{(aplicação da regra } S \to aSb\text{)} \\
&\Rightarrow aaSbb && \text{(aplicação da regra } S \to aSb\text{)} \\
&\Rightarrow aabb && \text{(aplicação da regra } S \to \varepsilon\text{)}
\end{aligned}
$$

A estrutura dessa derivação pode ser visualizada por meio de uma árvore de derivação, representada na Figura 1, na qual a raiz corresponde ao símbolo inicial, os nós internos representam as variáveis substituídas e as folhas correspondem aos terminais que compõem a palavra gerada (Sipser, 2012).

FIGURA 1 – ÁRVORE DE DERIVAÇÃO DA PALAVRA $aabb$ NA GRAMÁTICA G



FONTE: Os autores (2026)

De modo geral, cada aplicação da regra $S \to aSb$ acrescenta um a à esquerda e um b à direita do símbolo S, enquanto a regra $S \to \varepsilon$ encerra a derivação eliminando a variável remanescente. Conclui-se que $L(G) = \{a^n b^n \mid n \geq 0\}$, linguagem que, conforme será discutido adiante, não pode ser gerada por nenhum mecanismo mais restrito que uma gramática livre de contexto (Lewis; Papadimitriou, 1998; Sipser, 2012).

Restrições impostas sobre a forma das regras de produção dão origem a diferentes classes de gramáticas e, consequentemente, a diferentes classes de linguagens. Essa estratificação, conhecida como Hierarquia de Chomsky, organiza as linguagens

em quatro categorias (irrestritas, sensíveis ao contexto, livres de contexto e regulares), cada uma associada a um modelo computacional específico (Lewis; Papadimitriou, 1998; Vieira, 2004). O Quadro 1 sintetiza essa classificação, evidenciando a forma das regras de produção admitidas em cada classe e o modelo computacional que reconhece a linguagem correspondente.

QUADRO 1 – HIERARQUIA DE CHOMSKY

Tipo	**Classe de Linguagem**	**Forma das Regras**	**Modelo Computacional**
0	Recursivamente enumeráveis	$\alpha \to \beta$	Máquina de Turing
1	Sensíveis ao contexto	$\alpha A \beta \to \alpha \gamma \beta$	Autômato linearmente limitado
2	Livres de contexto	$A \to \gamma$	Autômato com pilha
3	Regulares	$A \to aB$ ou $A \to a$	Autômato finito

FONTE: Adaptado de Vieira (2004, p. 222–228) e Lewis e Papadimitriou (1998, p. 271–272)

As gramáticas regulares (Tipo 3), em particular, geram exatamente a classe das linguagens reconhecidas pelos autômatos finitos, formalismo apresentado nas próximas seções.

2.3 AUTÔMATO FINITO DETERMINÍSTICO

Conforme indicado ao final da seção anterior, as gramáticas regulares (Tipo 3 da Hierarquia de Chomsky) descrevem exatamente a classe das linguagens reconhecidas pelo modelo computacional mais simples: o autômato finito. Esta seção apresenta a formalização do autômato finito determinístico (AFD), modelo que serve como ponto de partida para o estudo dos reconhecedores de linguagens regulares e que constitui o objeto central da ferramenta proposta neste trabalho.

Intuitivamente, um AFD é uma máquina abstrata composta por um número finito de estados, interligados por transições rotuladas com símbolos de um alfabeto. A máquina inicia sua operação em um estado especial, denominado estado inicial, e processa uma cadeia de entrada lendo seus símbolos um a um, da esquerda para a direita. A cada símbolo lido, o autômato transita para um novo estado, conforme determinado pela rotulação das arestas. Encerrado o processamento da cadeia, se o autômato encontrar-se em um dos estados designados como estados finais (ou de aceitação), diz-se que a cadeia foi aceita. Caso contrário, é rejeitada (Hopcroft; Motwani; Ullman, 2006; Sipser, 2012). O termo *determinístico* reflete a característica

de que, para cada combinação de estado atual e símbolo de entrada, existe uma única transição possível.

Formalmente, um autômato finito determinístico é uma quíntupla $M = (Q, \Sigma, \delta, q_0, F)$, na qual (Hopcroft; Motwani; Ullman, 2006; Sipser, 2012):

- Q é um conjunto finito de estados;
- Σ é um alfabeto finito de símbolos de entrada;
- $\delta : Q \times \Sigma \to Q$ é a função de transição, que associa a cada par formado por um estado e um símbolo o estado para o qual o autômato transita;
- $q_0 \in Q$ é o estado inicial;
- $F \subseteq Q$ é o conjunto de estados finais (ou de aceitação).

A função δ descreve o comportamento do autômato ao ler um único símbolo. Para representar o processamento de uma cadeia inteira, define-se sua extensão recursiva $\hat{\delta} : Q \times \Sigma^* \to Q$, dada por (Hopcroft; Motwani; Ullman, 2006):

$$\hat{\delta}(q, \varepsilon) = q,$$
$$\hat{\delta}(q, wa) = \delta(\hat{\delta}(q, w), a),$$

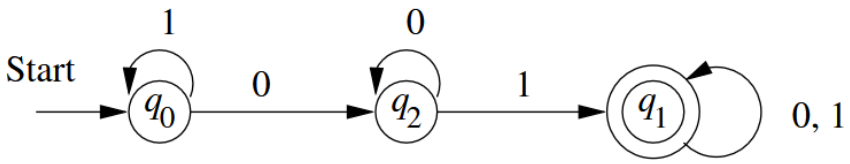
em que $w \in \Sigma^*$ e $a \in \Sigma$. A função estendida indica, portanto, o estado em que o autômato se encontra após processar uma cadeia w a partir do estado q. A linguagem reconhecida (ou aceita) por um AFD M, denotada $L(M)$, é o conjunto de todas as cadeias que conduzem o autômato, partindo do estado inicial, a algum estado de aceitação:

$$L(M) = \{\, w \in \Sigma^* \mid \hat{\delta}(q_0, w) \in F \,\}.$$

As linguagens reconhecidas por algum AFD são denominadas linguagens regulares (Sipser, 2012).

A representação gráfica de um AFD é feita por meio de um diagrama de estados, no qual os estados são representados por círculos, as transições por arestas rotuladas com os símbolos correspondentes, o estado inicial é apontado por uma seta sem origem e os estados de aceitação são marcados com círculos duplos. A título de exemplo, considere o AFD apresentado na Figura 2, que reconhece a linguagem das cadeias sobre $\{0, 1\}$ que contêm 01 como subcadeia (Hopcroft; Motwani; Ullman, 2006).

FIGURA 2 – AFD QUE RECONHECE CADEIAS CONTENDO 01 COMO SUBCADEIA



FONTE: Hopcroft, Motwani e Ullman (2006, p. 48)

Formalmente, o autômato da Figura 2 é descrito como $M = (Q, \Sigma, \delta, q_0, F)$, em que $Q = \{q_0, q_1, q_2\}$, $\Sigma = \{0, 1\}$, $F = \{q_1\}$ e a função δ é dada pelas transições $\delta(q_0, 0) = q_2$, $\delta(q_0, 1) = q_0$, $\delta(q_2, 0) = q_2$, $\delta(q_2, 1) = q_1$, $\delta(q_1, 0) = q_1$ e $\delta(q_1, 1) = q_1$. Cada estado codifica uma informação distinta sobre o histórico da leitura: q_0 indica que ainda não foi lido nenhum 0 (ou que o último símbolo lido foi 1), q_2 indica que o último símbolo lido foi 0 e q_1 indica que a subcadeia 01 já foi identificada (Hopcroft; Motwani; Ullman, 2006).

A título de ilustração, ao processar a cadeia 1101, o autômato realiza a sequência de transições $q_0 \xrightarrow{1} q_0 \xrightarrow{1} q_0 \xrightarrow{0} q_2 \xrightarrow{1} q_1$, atingindo o estado de aceitação q_1, de modo que a cadeia é aceita. Por outro lado, ao processar a cadeia 111, o autômato permanece em q_0 ao longo de toda a leitura e a cadeia é rejeitada por não terminar em estado final. Verifica-se que $L(M)$ corresponde exatamente ao conjunto das cadeias sobre $\{0, 1\}$ que contêm a subcadeia 01, em conformidade com a especificação informal apresentada.

2.4 AUTÔMATO FINITO NÃO-DETERMINÍSTICO

O autômato finito determinístico apresentado na seção anterior caracteriza-se por possuir, para cada combinação de estado e símbolo de entrada, uma única transição definida. Em diversas situações, no entanto, é mais natural descrever um reconhecedor permitindo que, em certos estados, mais de uma transição esteja disponível para o mesmo símbolo, ou ainda que, em outros, não haja qualquer transição definida (Hopcroft; Motwani; Ullman, 2006; Sipser, 2012). Tal flexibilidade dá origem ao autômato finito não-determinístico (AFN), modelo que, embora distinto em comportamento, reconhece a mesma classe de linguagens que os AFDs.

A diferença essencial em relação ao AFD reside na natureza da função de transição. Enquanto, em um AFD, a aplicação de δ a um par estado-símbolo retorna

um único estado, em um AFN ela retorna um conjunto de estados, possivelmente vazio. Isso significa que, ao processar um símbolo a partir de um dado estado, o AFN pode prosseguir por qualquer um dos estados indicados (caso haja mais de um) ou travar (caso o conjunto seja vazio). Intuitivamente, imagina-se que o autômato investiga simultaneamente todas as escolhas possíveis: o caminho computacional que, ao final da leitura, atinge um estado de aceitação determina a aceitação da cadeia, mesmo que outros caminhos tenham travado ou terminado em estados não-finais (Sipser, 2012).

Formalmente, um autômato finito não-determinístico é uma quíntupla $N = (Q, \Sigma, \delta, q_0, F)$, na qual (Hopcroft; Motwani; Ullman, 2006; Sipser, 2012):

- Q é um conjunto finito de estados;
- Σ é um alfabeto finito de símbolos de entrada;
- $\delta : Q \times \Sigma \to \mathcal{P}(Q)$ é a função de transição, que associa a cada par estado-símbolo um subconjunto de Q, sendo $\mathcal{P}(Q)$ o conjunto das partes de Q;
- $q_0 \in Q$ é o estado inicial;
- $F \subseteq Q$ é o conjunto de estados finais.

A extensão da função de transição a cadeias, denotada $\hat{\delta}$, é definida recursivamente de modo análogo ao caso determinístico, levando em consideração que cada passo pode produzir múltiplos estados (Hopcroft; Motwani; Ullman, 2006):

$$\hat{\delta}(q, \varepsilon) = \{q\},$$
$$\hat{\delta}(q, wa) = \bigcup_{p \in \hat{\delta}(q,w)} \delta(p, a),$$

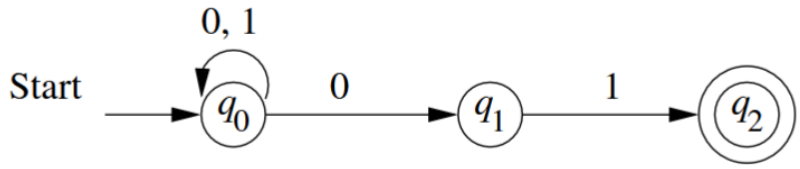
em que $w \in \Sigma^*$ e $a \in \Sigma$. A função estendida indica, portanto, o conjunto de todos os estados em que o autômato pode encontrar-se após processar a cadeia w a partir do estado q. A linguagem aceita por um AFN N, denotada $L(N)$, é o conjunto das cadeias para as quais existe ao menos um caminho computacional terminando em estado de aceitação:

$$L(N) = \{\, w \in \Sigma^* \mid \hat{\delta}(q_0, w) \cap F \neq \varnothing \,\}.$$

Em outras palavras, basta que uma das possíveis sequências de transições conduza a um estado final para que a cadeia seja aceita (Hopcroft; Motwani; Ullman, 2006; Sipser, 2012).

A título de exemplo, considere o AFN apresentado na Figura 3, que reconhece a linguagem das cadeias sobre $\{0, 1\}$ que terminam em 01 (Hopcroft; Motwani; Ullman, 2006).

FIGURA 3 – AFN QUE RECONHECE CADEIAS TERMINADAS EM 01



FONTE: Hopcroft, Motwani e Ullman (2006, p. 56)

Esse autômato é descrito formalmente como $N = (\{q_0, q_1, q_2\}, \{0, 1\}, \delta, q_0, \{q_2\})$, em que a função de transição é dada por $\delta(q_0, 0) = \{q_0, q_1\}$, $\delta(q_0, 1) = \{q_0\}$, $\delta(q_1, 1) = \{q_2\}$ e os demais valores correspondem ao conjunto vazio. As duas características que evidenciam o não-determinismo ficam aparentes nessa especificação: o estado q_0 admite duas transições distintas para o símbolo 0 (o autômato pode permanecer em q_0 ou avançar para q_1), enquanto os estados q_1 e q_2 apresentam transições ausentes para certos símbolos.

Para ilustrar o funcionamento, considere o processamento da cadeia 001. Partindo de $\hat{\delta}(q_0, \varepsilon) = \{q_0\}$, calcula-se sucessivamente $\hat{\delta}(q_0, 0) = \{q_0, q_1\}$, $\hat{\delta}(q_0, 00) = \{q_0, q_1\}$ e $\hat{\delta}(q_0, 001) = \{q_0, q_2\}$. Como $q_2 \in F$, a cadeia é aceita. Por outro lado, ao processar a cadeia 010, obtém-se ao final $\hat{\delta}(q_0, 010) = \{q_0, q_1\}$, conjunto que não contém o estado final q_2, de modo que a cadeia é rejeitada.

Uma extensão usual desse modelo admite transições ε (transições vazias), que permitem ao autômato mudar de estado sem consumir nenhum símbolo da entrada (Hopcroft; Motwani; Ullman, 2006; Sipser, 2012). Nesse modelo, denominado AFN-ε, a função de transição passa a ser $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \to \mathcal{P}(Q)$. Para tratar essas transições, define-se o ε-fecho de um estado q, denotado $E(q)$, como o conjunto de todos os estados alcançáveis a partir de q seguindo apenas transições ε, incluindo o próprio q (Hopcroft; Motwani; Ullman, 2006). A função estendida aplica o ε-fecho no início e após a leitura de cada símbolo:

$$\hat{\delta}(q, \varepsilon) = E(q),$$

$$\hat{\delta}(q, wa) = \bigcup_{p \in \hat{\delta}(q,w)} \bigcup_{r \in \delta(p,a)} E(r),$$

e a cadeia w é aceita, como antes, quando $\hat{\delta}(q_0, w) \cap F \neq \varnothing$. As transições ε não ampliam o poder de reconhecimento do modelo: todo AFN-ε pode ser convertido em um AFN sem transições ε e, portanto, em um AFD que reconhece a mesma linguagem (Hopcroft; Motwani; Ullman, 2006). Seu interesse é sobretudo prático, pois simplificam a descrição e a composição de autômatos. Um exemplo é o AFN-ε da Listagem 6.2, no Capítulo 6, que reconhece a linguagem a^*b^* com uma única transição ε entre um estado que lê os símbolos a e outro que lê os símbolos b.

Embora o não-determinismo confira maior flexibilidade aparente na descrição de linguagens, demonstra-se que toda linguagem reconhecida por um AFN também pode ser reconhecida por algum AFD (Hopcroft; Motwani; Ullman, 2006; Sipser, 2012). Esse resultado, fundamentado na chamada construção de subconjuntos, é apresentado na seção seguinte e justifica o fato de ambos os modelos definirem exatamente a mesma classe de linguagens: as linguagens regulares.

2.5 EQUIVALÊNCIA ENTRE AFD E AFN

Embora o autômato finito não-determinístico ofereça maior flexibilidade na descrição de linguagens, demonstra-se que essa flexibilidade não amplia a classe de linguagens reconhecíveis. Mais precisamente, vale o seguinte resultado central da teoria dos autômatos: uma linguagem L é reconhecida por algum AFD se, e somente se, é reconhecida por algum AFN (Hopcroft; Motwani; Ullman, 2006; Sipser, 2012). Em outras palavras, AFDs e AFNs são modelos equivalentes em poder de reconhecimento, ainda que possam diferir significativamente em número de estados e em facilidade de projeto.

A demonstração desse resultado é constituída por duas direções. A primeira, e mais imediata, consiste em observar que todo AFD pode ser interpretado como um AFN: basta tomar a função de transição do AFD e, para cada par estado-símbolo, considerar o conjunto unitário formado pelo único destino existente. Como o AFN resultante simula passo a passo o AFD original, ambos reconhecem a mesma linguagem (Hopcroft; Motwani; Ullman, 2006). A segunda direção, conhecida como construção de subconjuntos (*subset construction*), é menos trivial e fornece um procedimento algorítmico para converter qualquer AFN em um AFD equivalente.

A construção de subconjuntos parte da seguinte ideia: em um AFN, após ler uma

cadeia w, o autômato pode encontrar-se em qualquer um dos estados pertencentes a $\hat{\delta}(q_0, w)$. O AFD equivalente, por sua vez, simula esse comportamento mantendo, em cada um de seus estados, *o conjunto de estados do AFN* em que a computação poderia estar a partir da leitura realizada até aquele momento. Os estados do AFD são, portanto, subconjuntos do conjunto de estados do AFN (Hopcroft; Motwani; Ullman, 2006).

Formalmente, dado um AFN $N = (Q_N, \Sigma, \delta_N, q_{0,N}, F_N)$, constrói-se o AFD equivalente $D = (Q_D, \Sigma, \delta_D, q_{0,D}, F_D)$ por meio das seguintes definições (Hopcroft; Motwani; Ullman, 2006):

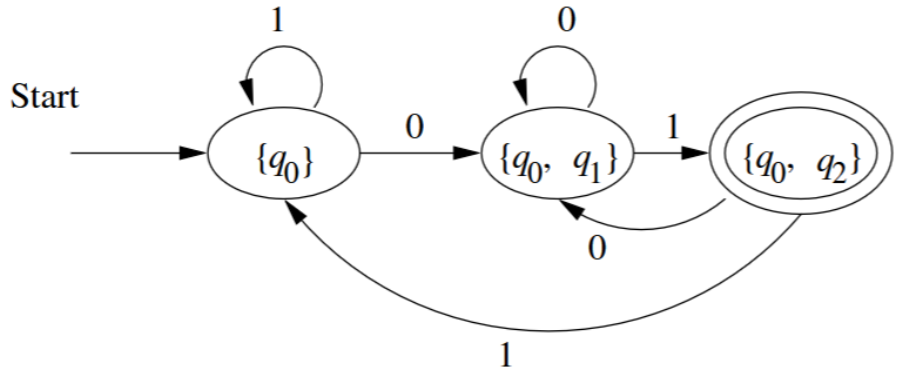
- $Q_D = \mathcal{P}(Q_N)$, ou seja, cada estado do AFD é um subconjunto de estados do AFN;
- $q_{0,D} = \{q_{0,N}\}$, isto é, o estado inicial do AFD corresponde ao conjunto que contém apenas o estado inicial do AFN;
- $F_D = \{\, S \in Q_D \mid S \cap F_N \neq \varnothing \,\}$, ou seja, um subconjunto é considerado estado de aceitação do AFD se contiver ao menos um estado de aceitação do AFN;
- $\delta_D(S, a) = \bigcup_{p \in S} \delta_N(p, a)$ para cada $S \in Q_D$ e cada $a \in \Sigma$, isto é, a transição do AFD a partir de um subconjunto S é a união das transições que o AFN executaria a partir de cada estado em S.

Embora $\mathcal{P}(Q_N)$ contenha $2^{|Q_N|}$ elementos no caso geral, em geral apenas uma pequena fração desses subconjuntos é efetivamente alcançável a partir do estado inicial. Na prática, o procedimento é aplicado de forma incremental: parte-se de $\{q_{0,N}\}$ e, a cada passo, computam-se as transições para os subconjuntos ainda não explorados, até que nenhum novo subconjunto seja gerado. A correção da construção é assegurada pelo seguinte teorema, demonstrado em Hopcroft, Motwani e Ullman (2006): *uma linguagem $L \subseteq \Sigma^*$ é reconhecida por algum AFN se, e somente se, é reconhecida por algum AFD.*

A título de ilustração, considere novamente o AFN da Figura 3, que reconhece as cadeias sobre $\{0, 1\}$ terminadas em 01. Aplicando a construção de subconjuntos a esse autômato, parte-se do estado inicial $\{q_0\}$ e calculam-se as transições conforme o procedimento descrito. Tem-se, por exemplo, $\delta_D(\{q_0\}, 0) = \delta_N(q_0, 0) = \{q_0, q_1\}$ e

$\delta_D(\{q_0\}, 1) = \{q_0\}$. Continuando o processo, geram-se sucessivamente os demais subconjuntos alcançáveis até que o AFD se estabilize. O resultado final é apresentado na Figura 4.

FIGURA 4 – AFD CONSTRUÍDO A PARTIR DO AFN DA FIGURA 3 PELA CONSTRUÇÃO DE SUBCONJUNTOS



FONTE: Hopcroft, Motwani e Ullman (2006, p. 63)

Observa-se que o AFD resultante possui apenas três estados ($\{q_0\}$, $\{q_0, q_1\}$ e $\{q_0, q_2\}$), ainda que o conjunto das partes de Q_N contivesse oito subconjuntos possíveis. O estado $\{q_0, q_2\}$ é o único estado de aceitação, pois é o único subconjunto acessível que contém q_2, estado final do AFN de origem. Verifica-se diretamente que $L(D) = L(N)$, isto é, o AFD construído reconhece exatamente a mesma linguagem do AFN de partida (Hopcroft; Motwani; Ullman, 2006).

Vale observar, por fim, que, no pior caso, a construção de subconjuntos pode produzir um AFD com 2^n estados a partir de um AFN com n estados, ainda que, na prática, o número de estados acessíveis costume ser comparável ao do AFN original (Hopcroft; Motwani; Ullman, 2006). Essa diferença, contudo, não altera a equivalência em poder expressivo: ambos os modelos definem exatamente a classe das linguagens regulares.

2.6 REPRESENTAÇÕES DE AUTÔMATOS

A definição formal de autômato finito como uma quíntupla, apresentada nas seções anteriores, é suficiente para caracterizar matematicamente o objeto, mas não é necessariamente a forma mais prática de descrevê-lo no dia a dia. Na literatura e em ferramentas educacionais, são utilizadas três representações distintas e comple-

mentares, cada uma adequada a um propósito específico (Hopcroft; Motwani; Ullman, 2006; Sipser, 2012). Esta seção apresenta essas representações, evidenciando as vantagens e limitações de cada uma.

A primeira forma é a notação formal, em que o autômato é descrito explicitamente como a quíntupla $M = (Q, \Sigma, \delta, q_0, F)$, listando-se cada um de seus componentes. Trata-se da representação mais precisa e desambígua, sendo indispensável em demonstrações matemáticas e em provas de equivalência. Por outro lado, sua leitura direta é trabalhosa: a função de transição δ, em particular, costuma ser apresentada como uma enumeração de pares $\delta(q, a) = q'$, que rapidamente se torna extensa à medida que cresce o número de estados (Hopcroft; Motwani; Ullman, 2006).

A segunda forma é o diagrama de estados, já utilizado nas Figuras 2 e 3 das seções anteriores. Nessa representação, cada estado corresponde a um nó (círculo simples para estados comuns ou círculo duplo para estados de aceitação), as transições são arestas rotuladas com os símbolos do alfabeto, e o estado inicial é apontado por uma seta sem origem (Hopcroft; Motwani; Ullman, 2006; Sipser, 2012). O diagrama de estados é o formato preferido em contextos didáticos e em ferramentas como o JFLAP (Rodger; Finley, 2006), pois permite que o leitor reconheça visualmente padrões como ciclos, estados de absorção e simetrias. Sua principal limitação é a dificuldade de manipulação textual: o diagrama é uma imagem e, portanto, não pode ser facilmente versionado, comparado ou processado por ferramentas automatizadas.

A terceira forma é a tabela de transição, que organiza a função δ em uma matriz bidimensional cujas linhas correspondem aos estados e cujas colunas correspondem aos símbolos do alfabeto (Hopcroft; Motwani; Ullman, 2006). O Quadro 2 ilustra essa representação para o AFD da Figura 2, que reconhece cadeias contendo 01 como subcadeia. A seta ($\rightarrow$) marca o estado inicial e o asterisco ($*$) destaca o estado de aceitação, conforme convenção adotada por Hopcroft, Motwani e Ullman (2006).

QUADRO 2 – TABELA DE TRANSIÇÃO DO AFD QUE RECONHECE CADEIAS CONTENDO 01

	0	**1**
$\rightarrow q_0$	q_2	q_0
q_2	q_2	q_1
$*q_1$	q_1	q_1

FONTE: Adaptado de Hopcroft, Motwani e Ullman (2006, p. 49)

A leitura da tabela é direta: cada célula (q, a) contém o estado $\delta(q, a)$ para o qual o autômato transita ao ler o símbolo a a partir do estado q. Essa representação combina precisão formal e leitura sistemática, sendo compacta, permitindo verificar facilmente se a função de transição está completa (no caso do AFD, todas as células devem estar preenchidas) e sendo particularmente adequada para o processamento programático, pois pode ser diretamente codificada como uma matriz ou estrutura de dados equivalente (Hopcroft; Motwani; Ullman, 2006; Sipser, 2012). Para AFNs, a tabela de transição segue o mesmo formato, com a diferença de que cada célula contém um *conjunto* de estados em vez de um único estado, podendo, inclusive, conter o conjunto vazio quando não houver transição definida (Hopcroft; Motwani; Ullman, 2006).

As três representações apresentadas (notação formal, diagrama de estados e tabela de transição) são complementares e equivalentes entre si, descrevendo o mesmo objeto matemático sob perspectivas distintas. A escolha entre elas depende do contexto de uso: o rigor formal favorece a notação como quíntupla; a comunicação didática favorece o diagrama; a manipulação computacional favorece a tabela. A ferramenta proposta neste trabalho explora essa equivalência ao oferecer uma quarta forma de representação, baseada em uma linguagem de domínio específico textual, que combina a precisão da notação formal com a estruturação tabular, permitindo simultaneamente a renderização do diagrama de estados e o processamento programático da descrição.

2.7 SIMULAÇÃO DE AUTÔMATOS

A definição formal de um autômato finito, vista nas seções anteriores, descreve sob a forma de quíntupla os elementos que compõem a máquina, mas não esclarece, por si só, como verificar se uma cadeia específica é aceita ou rejeitada. Essa verificação, conhecida como simulação, consiste em executar passo a passo a função de transição δ a partir do estado inicial, consumindo cada símbolo da cadeia de entrada e observando se a configuração final corresponde a um estado de aceitação (Hopcroft; Motwani; Ullman, 2006; Sipser, 2012). Em termos pedagógicos, a simulação cumpre um papel análogo ao da execução de um programa em uma linguagem de programação, pois permite que o estudante observe o comportamento do autômato projetado e confronte-o com o comportamento esperado (Morazán; Antunez, 2014).

Morazán e Antunez (2014) argumentam que disciplinas introdutórias de linguagens formais, tradicionalmente, exigem que estudantes projetem máquinas e provem sua correção sem oferecer mecanismos para experimentar com os artefatos produzidos. Os autores defendem que essa ausência de retroalimentação imediata contraria a forma como o estudante de Ciência da Computação aprende a programar, em que compiladores e interpretadores fornecem mensagens de erro e advertências durante o desenvolvimento. A consequência observada é que estudantes inexperientes submetem com frequência soluções incorretas, mesmo para exercícios relativamente simples, e dedicam esforço a tentativas de prova de correção sobre projetos defeituosos (Morazán; Antunez, 2014). Para mitigar esse problema, os autores propõem que a teoria da computação seja ensinada com ferramentas capazes de *construir* computações, isto é, ambientes em que a descrição formal possa ser traduzida em uma representação executável e submetida a testes.

Formalmente, simular um AFD $M = (Q, \Sigma, \delta, q_0, F)$ sobre uma cadeia $w = a_1 a_2 \ldots a_n \in \Sigma^*$ corresponde a calcular a sequência de configurações $q_0, q_1, \ldots, q_n$, em que $q_i = \delta(q_{i-1}, a_i)$ para $1 \leq i \leq n$. A cadeia w é aceita se, e somente se, $q_n \in F$ (Hopcroft; Motwani; Ullman, 2006). Para um AFD, a simulação é determinística e linear: cada símbolo lido conduz a exatamente um próximo estado, de modo que o custo computacional cresce proporcionalmente ao comprimento da cadeia. Essa propriedade torna o AFD particularmente adequado para implementações eficientes, pois pode ser realizado por uma estrutura de dados simples, como uma tabela de transição indexada pelo par (estado, símbolo) (Sipser, 2012).

A simulação de um AFN, em contraste, exige tratamento adicional, pois a função de transição δ pode levar um mesmo par (estado, símbolo) a um conjunto de estados, e transições ε permitem mudança de estado sem consumir símbolos da entrada. Conforme descrito por Morazán e Antunez (2014) ao apresentar a função *apply-sm* da biblioteca FSM, a simulação de uma máquina não determinística é realizada por meio de uma busca em largura sobre todos os caminhos possíveis que o autômato pode seguir ao consumir a cadeia. Em cada passo, o simulador mantém o conjunto de estados alcançáveis e, ao final da entrada, verifica se algum desses estados pertence ao conjunto de aceitação. Alternativamente, conforme Hopcroft, Motwani e Ullman (2006), o AFN pode ser convertido previamente em um AFD equivalente pelo algoritmo

de construção de subconjuntos, transferindo a simulação para o caso determinístico ao custo de uma possível expansão exponencial do número de estados.

Além de produzir um veredito de aceitação ou rejeição, uma ferramenta de simulação útil para o ensino deve expor o *caminho percorrido* pelo autômato, isto é, a sequência de configurações intermediárias atravessadas durante o processamento da cadeia. Morazán e Antunez (2014) denominam essa funcionalidade *show-transitions* e destacam sua importância para que o estudante identifique exatamente em que ponto sua máquina diverge do comportamento esperado. Essa visualização do *trace* de execução substitui o exercício manual de acompanhar transições em papel, propenso a erros, por uma observação direta do comportamento da máquina, em consonância com a prática de depuração comum no desenvolvimento de *software*.

A relevância da simulação ganha ainda mais destaque quando se considera o fenômeno descrito por Morazán e Antunez (2014) de que estudantes, sem ambiente de teste, frequentemente partem para demonstrações formais sobre projetos incorretos, com perda de tempo e desmotivação como consequência. Esse argumento é central para o presente trabalho: a ferramenta proposta busca oferecer simulação visual interativa diretamente acoplada à descrição textual do autômato, de forma que o estudante possa, a partir de uma única especificação, examinar a estrutura do diagrama de estados, executar cadeias de teste e acompanhar a evolução das configurações, recebendo retroalimentação imediata sobre o comportamento da máquina.

Cabe ainda diferenciar dois sentidos do termo simulação que aparecem na literatura. Em sentido estrito, simular um autômato é executar sua função de transição sobre uma entrada específica, como descrito acima. Em sentido mais amplo, ferramentas como o JFLAP (Rodger; Finley, 2006) e a biblioteca FSM (Morazán; Antunez, 2014) oferecem também a possibilidade de *testar* um autômato com múltiplas cadeias geradas aleatoriamente, comparar duas máquinas quanto à equivalência de suas linguagens e visualizar transformações construtivas, como a conversão de AFN em AFD. Essas operações estendem a noção básica de simulação para um conjunto mais rico de recursos didáticos, ao qual se pode atribuir o termo *ambiente de experimentação*. A ferramenta proposta neste trabalho posiciona-se nesse espectro, oferecendo, em primeiro plano, a simulação visual passo a passo de autômatos finitos descritos por meio de uma linguagem de domínio específico, conforme será detalhado no Capítulo 7.

2.8 APLICAÇÕES DE AUTÔMATOS

Apesar de serem objetos teóricos, autômatos finitos não são uma abstração restrita ao ambiente acadêmico. Hopcroft, Motwani e Ullman (2006) destacam que esse modelo computacional sustenta o funcionamento de diversas categorias de *software* e *hardware* utilizadas no dia a dia, sempre que um sistema precisa registrar, em um número finito de configurações, a parte relevante de seu histórico para decidir qual ação tomar diante da próxima entrada. Esta seção apresenta as principais áreas de aplicação descritas pelos autores e estabelece a ponte entre os conceitos formais discutidos até aqui e os capítulos seguintes deste trabalho.

Hopcroft, Motwani e Ullman (2006) agrupam as aplicações dos autômatos finitos em quatro grandes categorias:

- Projeto e verificação de circuitos digitais. Sistemas digitais com um número finito de estados internos podem ser modelados diretamente como autômatos, permitindo que o comportamento do circuito seja descrito de forma rigorosa e verificado antes da implementação física. Cada estado do autômato corresponde a uma configuração possível do circuito, e cada transição corresponde à resposta do *hardware* a um sinal de entrada (Hopcroft; Motwani; Ullman, 2006).

- Análise léxica de compiladores. O componente responsável por dividir o texto-fonte em unidades léxicas (identificadores, palavras-chave, operadores e literais) é tipicamente implementado como um autômato finito. Hopcroft, Motwani e Ullman (2006) ilustram esse uso com um autômato cuja função é reconhecer a palavra-chave `then`: cada estado corresponde a um prefixo da palavra (vazio, t, th, the, $then$), e a sequência de transições percorrida pela cadeia de entrada determina se o lexema foi encontrado. Esse mesmo princípio se generaliza para o reconhecimento de qualquer conjunto finito de *tokens* descrito por expressões regulares.

- Busca de padrões em grandes volumes de texto. Sistemas que percorrem coleções extensas de documentos, páginas *web* ou arquivos em busca de palavras, expressões ou padrões mais complexos costumam compilar essas consultas em autômatos finitos, executando a varredura em tempo proporcional ao tamanho da entrada (Hopcroft; Motwani; Ullman, 2006). Utilitários clássicos do ambiente

Unix, como *grep*, e bibliotecas modernas de expressões regulares apoiam-se diretamente nessa equivalência entre expressões regulares e autômatos finitos.

- Verificação de sistemas com número finito de estados. Protocolos de comunicação, protocolos criptográficos e outros sistemas reativos podem ser modelados como autômatos para que propriedades como ausência de *deadlock*, alcançabilidade de estados ou conformidade com uma especificação sejam verificadas automaticamente (Hopcroft; Motwani; Ullman, 2006).

A presença pervasiva dos autômatos finitos nessas áreas decorre de uma característica central do modelo: o número fixo de estados permite que sua representação seja implementada com uma quantidade limitada de recursos, seja como um circuito de *hardware*, seja como uma estrutura de dados em memória (Hopcroft; Motwani; Ullman, 2006). Esse equilíbrio entre simplicidade e poder expressivo explica por que o modelo permanece relevante mesmo após décadas de evolução das ferramentas de *software*.

Para o presente trabalho, a aplicação de maior interesse é a análise léxica, pois constitui o elo direto entre a teoria dos autômatos finitos e a construção de compiladores e interpretadores para linguagens de programação. A linguagem de domínio específico proposta nesta monografia, descrita no Capítulo 6, é analisada lexicamente por um autômato finito gerado a partir das regras léxicas da própria linguagem, em uma aplicação concreta dos conceitos apresentados neste capítulo.

3 COMPILADORES E LINGUAGENS DE DOMÍNIO ESPECÍFICO

Este capítulo apresenta os fundamentos teóricos de compiladores e linguagens de domínio específico que embasam a ferramenta proposta. Inicia-se com os conceitos gerais de compiladores e sua distinção em relação a interpretadores (Seção 3.1), seguindo-se a descrição detalhada das três fases do *front-end*: análise léxica (Seção 3.2), análise sintática (Seção 3.3) e análise semântica (Seção 3.4). Em seguida, apresenta-se a Árvore Sintática Abstrata como estrutura intermediária central do processo de compilação (Seção 3.5). A partir desses fundamentos, introduz-se o conceito de Linguagem de Domínio Específico (Seção 3.6), discutem-se as principais ferramentas e abordagens para sua construção (Seção 3.7), trata-se do diagnóstico de erros como componente essencial de qualidade (Seção 3.8) e, por fim, examina-se o uso de DSLs no contexto do ensino de linguagens formais (Seção 3.9), perspectiva que motiva diretamente o desenvolvimento do presente trabalho.

3.1 CONCEITOS FUNDAMENTAIS DE COMPILADORES

Antes de tratar das fases específicas de análise que utilizam autômatos finitos, é necessário situar o que é um compilador e qual o seu papel no processo de tradução de programas. Esta seção apresenta a definição clássica de compilador, distingue-o do interpretador, que é o outro tipo mais comum de processador de linguagem, e descreve, em alto nível, a sequência de fases que constituem o pipeline de compilação. Os conceitos aqui introduzidos servirão de apoio às seções seguintes, que detalham cada fase do *front-end* de um compilador.

Um compilador é um programa que recebe como entrada um texto escrito em uma linguagem de programação, denominada linguagem fonte, e produz como saída um programa equivalente em outra linguagem, denominada linguagem objeto (ou linguagem alvo) (Aho *et al.*, 2008). A equivalência exigida nesse processo é semântica: o programa objeto deve produzir, sobre as mesmas entradas, exatamente os mesmos resultados que o programa fonte produziria, ainda que o faça em uma linguagem de nível mais baixo, em geral próxima da arquitetura da máquina de execução. Além de traduzir o programa, o compilador desempenha um segundo papel essencial, o de relatar, com precisão, eventuais erros detectados durante a tradução, fornecendo ao

programador o diagnóstico necessário para a correção do código fonte (Aho *et al.*, 2008).

A linguagem fonte costuma ser uma linguagem de alto nível, projetada para ser lida e escrita por humanos, ao passo que a linguagem objeto pode ser código de máquina diretamente executável, código de uma máquina virtual (como os *bytecodes* da JVM) ou ainda outra linguagem de alto nível, configurando-se nesse caso um *source-to-source compiler*, ou tradutor de fonte para fonte (Aho *et al.*, 2008). Independentemente da forma da saída, o cerne da tarefa permanece o mesmo, isto é, realizar uma tradução fiel preservando o significado original.

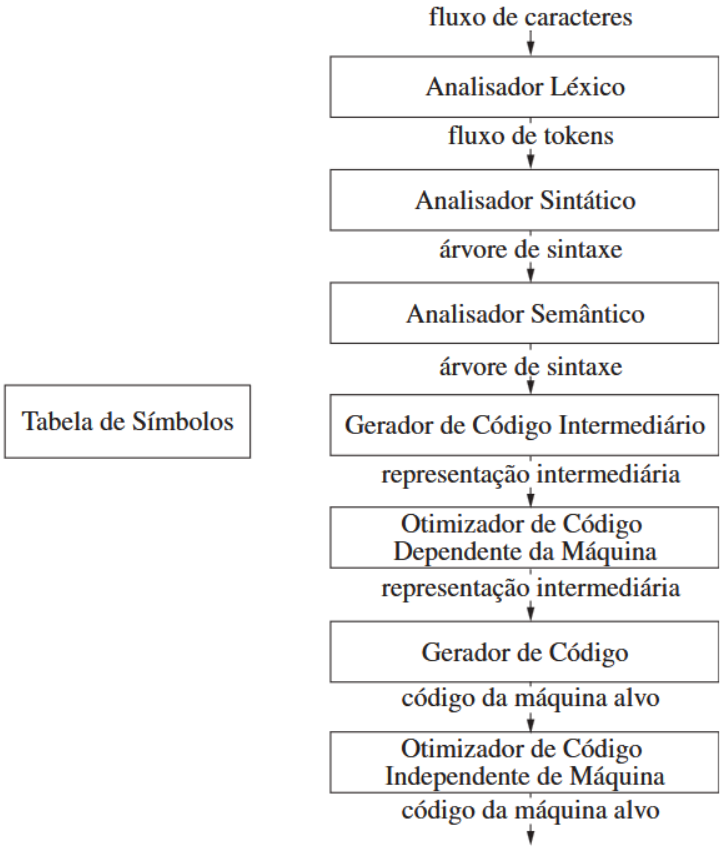
O compilador não é a única forma de processador de linguagem. Um interpretador, em vez de produzir um programa objeto que será posteriormente executado, realiza diretamente, sobre as entradas fornecidas pelo usuário, as operações descritas no programa fonte (Aho *et al.*, 2008). A diferença entre os dois modelos pode ser sintetizada pelo papel do programa fonte em tempo de execução: para o compilador, ele é apenas insumo de uma fase prévia, ausente no momento em que o programa objeto roda; para o interpretador, ele permanece presente durante toda a execução, sendo lido e processado instrução a instrução.

Cada abordagem apresenta vantagens distintas. O programa objeto produzido por um compilador costuma ser substancialmente mais rápido na execução, pois a tradução para a linguagem da máquina é feita uma única vez e o resultado é executado diretamente pelo processador. Por outro lado, o interpretador costuma oferecer um melhor diagnóstico de erros em tempo de execução, uma vez que tem acesso direto à estrutura do programa fonte enquanto o avalia (Aho *et al.*, 2008). As duas estratégias não são mutuamente exclusivas, e muitas implementações modernas combinam-nas em arranjos híbridos. O processador de Java, por exemplo, primeiro compila o programa fonte para uma forma intermediária portável (os *bytecodes*), que é então interpretada por uma máquina virtual. Certas máquinas virtuais ainda recompilam dinamicamente trechos do código para a linguagem da máquina hospedeira, técnica conhecida como compilação *just-in-time*, que busca obter simultaneamente a portabilidade da interpretação e o desempenho da compilação (Aho *et al.*, 2008).

Internamente, um compilador está longe de ser uma caixa-preta monolítica. Aho *et al.* (2008) descrevem o processo de compilação como uma sequência de fases, cada

uma transformando uma representação do programa fonte em outra, mais próxima do programa objeto, conforme ilustra a Figura 5. Essas fases agrupam-se em duas grandes etapas: a análise, responsável por compreender a estrutura e o significado do programa fonte, e a síntese, responsável por construir o programa objeto a partir dessa compreensão. A parte de análise é também denominada *front-end* do compilador, enquanto a parte de síntese constitui o *back-end*.

FIGURA 5 – FASES DE UM COMPILADOR



FONTE: Aho *et al.* (2008, p. 4)

A etapa de análise costuma ser subdividida em três fases. A análise léxica (também chamada de *scanning* ou leitura) percorre o fluxo de caracteres do programa fonte e os agrupa em sequências significativas, denominadas lexemas. Para cada lexema, produz um *token*, isto é, um par formado por um nome simbólico e, eventualmente, um valor de atributo associado. A análise sintática recebe o fluxo de *tokens* produzido pela fase léxica e o organiza em uma estrutura hierárquica, em geral uma árvore de sintaxe, que torna explícita a estrutura gramatical do programa. Por fim, a análise semântica percorre essa árvore para verificar a consistência de tipos e o cumprimento de outras regras da linguagem que não podem ser expressas pela gramática livre de contexto,

anotando a árvore com informações adicionais necessárias às fases seguintes (Aho *et al.*, 2008). Ao longo de todas essas fases, uma estrutura auxiliar denominada tabela de símbolos mantém informações sobre os identificadores declarados no programa, sendo consultada e atualizada conforme necessário.

Concluída a análise, a etapa de síntese gera, a partir da árvore anotada, uma representação intermediária, a qual pode ser submetida a uma fase opcional de otimização independente da máquina, com o objetivo de produzir código mais eficiente sem alterar a semântica do programa. Em seguida, o gerador de código traduz a representação intermediária para a linguagem da máquina alvo, e uma segunda fase opcional de otimização, dependente da máquina, refina o resultado explorando características específicas da arquitetura (Aho *et al.*, 2008). O programa objeto final é, portanto, produto de uma cadeia de transformações sucessivas, em que cada etapa preserva a semântica do programa original e adiciona ou remove informações conforme necessário.

Para os fins deste trabalho, interessa particularmente o *front-end* do compilador e, dentro dele, a fase de análise léxica, que constitui a aplicação direta dos autômatos finitos discutidos no Capítulo 2. Os mecanismos pelos quais expressões regulares são convertidas em autômatos para o reconhecimento de *tokens*, bem como as estratégias de implementação desses autômatos em código, são o tema da seção seguinte.

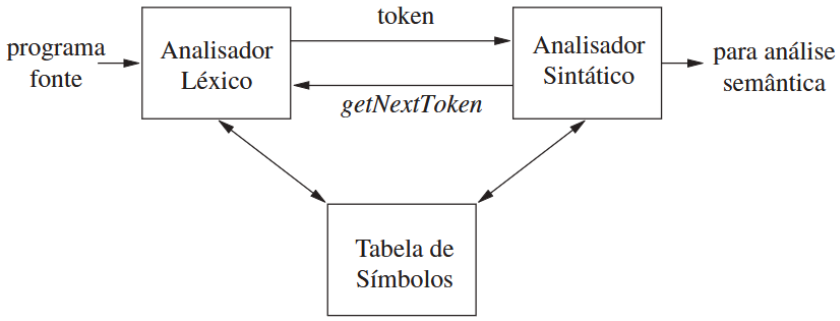
3.2 ANÁLISE LÉXICA

A análise léxica é a primeira fase do *front-end* de um compilador e tem como tarefa principal ler o fluxo bruto de caracteres do programa fonte, agrupá-lo em sequências significativas, denominadas lexemas, e produzir, para cada lexema, um *token* que será consumido pela fase seguinte de análise sintática (Aho *et al.*, 2008). Além desse papel central, o analisador léxico costuma ser responsável por tarefas auxiliares de pré-processamento, como a remoção de espaços em branco e comentários, o registro do número da linha em que cada lexema ocorre (informação essencial para a posterior emissão de mensagens de erro) e, em algumas implementações, a expansão de macros (Aho *et al.*, 2008).

A interação entre os dois primeiros analisadores costuma ser implementada de forma orientada à demanda, conforme ilustra a Figura 6. O analisador sintá-

tico invoca o analisador léxico por meio de uma chamada normalmente denominada `getNextToken`, e este, por sua vez, lê caracteres da entrada até identificar o próximo lexema, sobre o qual constrói o *token* a ser devolvido. Esse arranjo permite que o analisador léxico interaja também com a tabela de símbolos: quando reconhece um lexema correspondente a um identificador, insere ou consulta a entrada apropriada nessa tabela, de modo que as fases subsequentes tenham acesso uniforme às informações sobre os identificadores do programa (Aho *et al.*, 2008).

FIGURA 6 – INTERAÇÕES ENTRE O ANALISADOR LÉXICO E O ANALISADOR SINTÁTICO



FONTE: Aho *et al.* (2008, p. 70)

A separação entre análise léxica e análise sintática, embora pudesse, em princípio, ser dispensada, é justificada por três razões práticas, conforme apresentam Aho *et al.* (2008). A primeira é a simplicidade de projeto: tratar comentários e espaços em branco já no nível léxico evita poluir a gramática da fase sintática com construções acessórias. A segunda é a eficiência, pois técnicas especializadas de leitura, como o uso de *buffers* duplos com sentinelas, podem ser aplicadas exclusivamente à fase léxica, acelerando substancialmente o processamento. A terceira é a portabilidade, uma vez que peculiaridades do dispositivo de entrada e da codificação de caracteres ficam restritas ao analisador léxico, sem afetar as demais fases do compilador.

A discussão sobre análise léxica apoia-se em três termos relacionados, porém distintos. Um *token* é um par formado por um nome simbólico e, opcionalmente, um valor de atributo. O nome representa, de forma abstrata, uma classe de unidade léxica, como uma palavra-chave específica ou a categoria genérica de identificador. Um padrão é uma descrição da forma que os lexemas associados a um *token* podem assumir, em geral expressa por meio de uma expressão regular. Um lexema é uma sequência concreta de caracteres do programa fonte que casa com o padrão de um

token e é, por essa razão, classificada como uma instância desse *token* (Aho *et al.*, 2008). Por exemplo, em um programa em C que contenha o trecho `score = 3.14;`, a palavra `score` é um lexema que casa com o padrão de identificador e produz um *token id*, ao passo que `3.14` é um lexema que casa com o padrão de constante numérica e produz um *token number*.

Quando mais de um lexema pode casar com o mesmo padrão, como ocorre tipicamente com identificadores e constantes, o nome do *token* sozinho não basta para distinguir as ocorrências, e o analisador léxico precisa fornecer informação adicional às fases seguintes. Para isso, associa-se ao *token* um valor de atributo, que descreve o lexema específico encontrado. No caso de identificadores, esse atributo costuma ser um apontador para a entrada correspondente na tabela de símbolos, onde se mantêm informações como o nome, o tipo e a posição do identificador no programa fonte. No caso de constantes numéricas, o atributo pode ser o próprio valor numérico ou um apontador para uma representação interna desse valor (Aho *et al.*, 2008).

Eventualmente, o analisador léxico pode encontrar uma sequência de caracteres que não casa com nenhum dos padrões definidos. Em alguns casos, o erro só pode ser detectado por fases posteriores, pois o lexema é localmente válido, embora gramaticalmente inadequado: a sequência `fi`, em um programa C, é um identificador léxicamente válido, ainda que provavelmente seja um `if` grafado de forma equivocada (Aho *et al.*, 2008). Quando, porém, nenhum padrão casa com o prefixo restante da entrada, o analisador léxico precisa recorrer a alguma estratégia de recuperação. A mais simples, conhecida como recuperação em *modo de pânico*, consiste em descartar caracteres da entrada até que um *token* bem formado possa ser reconhecido. Estratégias mais elaboradas tentam reparar o erro por meio de uma única transformação local (remoção, inserção, substituição ou transposição de caracteres), partindo da hipótese, empírica, de que a maioria dos erros léxicos envolve um único caractere (Aho *et al.*, 2008).

Do ponto de vista de implementação, a especificação dos padrões dos *tokens* costuma ser feita por meio de expressões regulares, e o reconhecimento, por meio de autômatos finitos construídos a partir delas. Essa relação entre expressões regulares e autômatos, estabelecida no Capítulo 2, é exatamente o que torna a análise léxica uma aplicação direta da teoria de linguagens regulares: cada padrão corresponde a uma expressão regular, o conjunto desses padrões é convertido em um autômato finito

não determinístico (em geral pelo método de Thompson), o qual é então transformado em um autômato finito determinístico (pela construção de subconjuntos) capaz de reconhecer eficientemente os lexemas durante a varredura do programa fonte (Aho *et al.*, 2008). Essa cadeia, de *regex* a *token*, fornece a base teórica para os geradores automáticos de analisadores léxicos, como o Lex e o Flex, e também para bibliotecas modernas que empregam técnicas equivalentes em linguagens contemporâneas. A ferramenta proposta neste trabalho, descrita no Capítulo 6, faz uso direto dessa correspondência ao analisar lexicamente sua linguagem de domínio específico, em uma aplicação concreta dos conceitos discutidos nesta seção.

3.3 ANÁLISE SINTÁTICA

A análise sintática, também denominada *parsing*, é a segunda fase do *front-end* de um compilador e tem por finalidade verificar se a sequência de *tokens* produzida pela análise léxica forma uma cadeia válida segundo a gramática da linguagem fonte e, em caso afirmativo, organizá-la em uma estrutura hierárquica que torne explícita a estrutura gramatical do programa (Aho *et al.*, 2008). Essa estrutura é, em geral, uma árvore de derivação (também chamada de *parse tree*), na qual os nós internos representam as variáveis aplicadas durante a derivação e as folhas correspondem aos terminais que compõem a cadeia analisada. A Figura 7 ilustra esse conceito para a expressão $id + id * id$, segundo a gramática $E \rightarrow E + T \mid T$, $T \rightarrow T * F \mid F$ e $F \rightarrow (E) \mid id$, tradicionalmente utilizada em livros-texto sobre análise sintática (Aho *et al.*, 2008). Observe que a estrutura hierárquica captura naturalmente a precedência dos operadores, pois a multiplicação, mais profunda na árvore, é avaliada antes da adição.

A especificação da sintaxe das linguagens de programação utiliza, em geral, gramáticas livres de contexto, formalismo apresentado no Capítulo 2. A motivação dessa escolha é direta: as gramáticas regulares, embora suficientes para descrever os padrões léxicos, não conseguem expressar construções recursivamente aninhadas, como expressões parentizadas ou blocos de comandos balanceados, que são comuns em linguagens de programação. As gramáticas livres de contexto, por sua vez, capturam naturalmente essas construções por meio de regras recursivas, sendo o formalismo padrão para a especificação sintática de linguagens (Aho *et al.*, 2008).

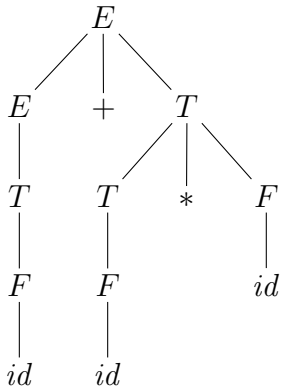


FIGURA 7 – ÁRVORE DE DERIVAÇÃO DA EXPRESSÃO $id + id * id$

FONTE: Os autores (2026)

Para programas bem formados, o analisador sintático constrói, ao menos conceitualmente, uma árvore de derivação que é então repassada às fases subsequentes do *front-end*, isto é, à análise semântica e à geração de código intermediário. Na prática, contudo, é comum que essa árvore não seja construída de forma explícita: as ações de tradução podem ser entremeadas com a análise propriamente dita, técnica conhecida como tradução dirigida por sintaxe, em que cada produção da gramática associa-se a um conjunto de ações semânticas executadas no momento em que a produção é reconhecida (Aho *et al.*, 2008). Esse arranjo permite, em muitos casos, integrar análise sintática, análise semântica e geração de código em um único módulo coeso.

Existem duas grandes famílias de algoritmos de análise sintática empregados em compiladores reais: a análise descendente (*top-down*) e a análise ascendente (*bottom-up*) (Aho *et al.*, 2008). Os métodos descendentes constroem a árvore de derivação da raiz para as folhas, partindo do símbolo inicial da gramática e tentando reescrevê-lo até obter a cadeia da entrada. Nessa família, destacam-se os analisadores preditivos e, em particular, os analisadores LL, em geral implementados manualmente como um conjunto de funções mutuamente recursivas, técnica conhecida como descida recursiva. Os métodos ascendentes, por sua vez, constroem a árvore das folhas para a raiz, agrupando progressivamente os *tokens* da entrada em estruturas maiores até alcançar o símbolo inicial. Os analisadores LR, mais expressivos que os LL, costumam ser construídos por meio de ferramentas automáticas, como Yacc e Bison, devido à complexidade das tabelas que utilizam (Aho *et al.*, 2008). Em ambas as estratégias, a entrada é consumida da esquerda para a direita, um símbolo de cada vez.

Espera-se de um analisador sintático mais do que a simples aceitação ou rejeição

da entrada. Ele deve sinalizar erros de forma clara e precisa, identificar o local de cada erro no programa fonte e, sempre que possível, recuperar-se de modo a permitir a detecção de erros subsequentes em uma única invocação do compilador (Aho *et al.*, 2008). Os autores discutem quatro estratégias gerais de recuperação de erros sintáticos. A recuperação em *modo pânico* descarta *tokens* da entrada até encontrar um *token* de sincronismo, como ponto-e-vírgula ou chave de fechamento, retomando a análise a partir desse ponto. É simples e robusta, embora possa ocultar erros próximos. A recuperação em nível de frase tenta corrigir o erro localmente por meio de pequenas substituições, inserções ou exclusões de *tokens*. A estratégia de produções de erro estende a gramática com regras que reconhecem construções erradas comuns, permitindo emitir mensagens de erro especializadas. Por fim, a correção global busca a menor sequência de transformações capaz de tornar a entrada sintaticamente válida. Embora teoricamente atraente, é dispendiosa demais para uso em compiladores reais.

Para os fins deste trabalho, a análise sintática é a fase responsável por reconhecer a estrutura gramatical da linguagem de domínio específico proposta, transformando a sequência de *tokens* produzida pela análise léxica em uma representação estruturada que descreve, de forma não ambígua, o autômato finito especificado pelo usuário. As decisões de projeto da gramática dessa linguagem, bem como o método de análise adotado em sua implementação, serão detalhados em capítulo posterior.

3.4 ANÁLISE SEMÂNTICA

A análise semântica é a terceira fase do *front-end* e tem por objetivo verificar a consistência do programa em relação a regras que não podem ser expressas pela gramática livre de contexto utilizada na fase sintática. Apoiando-se na árvore de derivação produzida pela análise sintática e nas informações armazenadas na tabela de símbolos, o analisador semântico identifica violações das regras de escopo, declaração e tipagem da linguagem fonte e, ao mesmo tempo, anota a árvore com informações adicionais, como tipos inferidos e conversões implícitas, que serão utilizadas pelas fases subsequentes de geração de código (Aho *et al.*, 2008).

A parcela mais visível do trabalho do analisador semântico é a verificação de tipos. Para realizá-la, o compilador atribui uma expressão de tipo a cada componente do programa fonte e verifica se essas expressões satisfazem o conjunto de regras lógicas

que constitui o sistema de tipos da linguagem. Exige-se tipicamente, por exemplo, que o índice usado para acessar um vetor seja inteiro, que os operandos de um operador aritmético tenham tipos compatíveis e que o valor retornado por uma função tenha o tipo declarado em sua assinatura (Aho *et al.*, 2008). Quando alguma dessas condições é violada, o compilador emite uma mensagem de erro de tipo, evitando que o programa avance para fases posteriores em estado inconsistente.

A verificação de tipos pode ser feita estaticamente, em tempo de compilação, ou dinamicamente, em tempo de execução. Um sistema de tipos é dito seguro quando a verificação estática realizada pelo compilador é suficiente para garantir que erros de tipo nunca ocorrerão durante a execução do programa objeto, dispensando, portanto, verificações dinâmicas dispendiosas. Uma implementação de linguagem é dita fortemente tipada (*strongly typed*) quando o compilador efetivamente assegura essa propriedade para todos os programas que aceita (Aho *et al.*, 2008). A verificação estática de tipos é, em geral, considerada uma das principais ferramentas para a detecção precoce de erros e tem aplicações que vão além do contexto restrito da compilação, como a verificação de *bytecodes* Java antes de sua execução em uma máquina virtual.

Aho *et al.* (2008) distinguem duas formas complementares de verificação de tipos. Na síntese de tipos, o tipo de uma expressão é construído a partir dos tipos de suas subexpressões, com base em regras locais. Essa abordagem exige, em geral, que todos os nomes utilizados sejam declarados antes de seu uso, de modo que o compilador disponha sempre da informação necessária à composição dos tipos. Na inferência de tipos, por outro lado, o tipo de uma construção é deduzido a partir do modo como ela é utilizada, por meio de variáveis de tipo e regras de unificação. Essa abordagem é típica de linguagens funcionais como ML e Haskell, que verificam os tipos sem exigir declarações explícitas. Em qualquer um dos dois modelos, o objetivo final é o mesmo, isto é, associar a cada expressão um tipo único e bem definido antes da geração de código.

Quando os operandos de uma operação possuem tipos diferentes, mas compatíveis em algum sentido, o compilador pode inserir automaticamente uma conversão de tipo, também denominada coerção, para uniformizar os operandos. Ao avaliar a soma de um inteiro com um número de ponto flutuante, por exemplo, é comum que o inteiro seja convertido para a representação de ponto flutuante antes da operação (Aho *et al.*,

2008). Essas conversões são denominadas implícitas quando inseridas pelo compilador sem intervenção do programador, e explícitas (ou *casts*) quando declaradas no próprio código fonte. As regras que regem cada tipo de conversão variam de linguagem para linguagem e integram a especificação semântica da linguagem fonte.

A verificação de tipos é, em geral, a parcela mais elaborada da análise semântica, mas não é a única. O analisador semântico também é responsável por verificar regras de escopo, garantindo que cada uso de um identificador corresponda a uma declaração visível no contexto em questão, por detectar declarações duplicadas em um mesmo escopo, por validar a unicidade de rótulos e por assegurar que estruturas como condicionais, laços e retornos respeitem as restrições impostas pela linguagem. Em linguagens com sobrecarga de funções ou operadores, cabe-lhe ainda escolher, entre as várias definições disponíveis para um mesmo nome, aquela cuja assinatura é compatível com o contexto de uso (Aho *et al.*, 2008).

Para os fins deste trabalho, a análise semântica desempenha um papel particular, pois a linguagem de domínio específico proposta descreve estruturas matemáticas (autômatos finitos) cuja correção depende de um conjunto de condições estruturais que vão além da sintaxe. Cabe à fase semântica verificar, por exemplo, se todos os estados referenciados em transições foram previamente declarados, se o estado inicial e cada estado final pertencem ao conjunto de estados, se cada símbolo utilizado em uma transição pertence ao alfabeto especificado e, no caso de autômatos determinísticos, se não existem transições conflitantes a partir de um mesmo estado para um mesmo símbolo. Essas verificações, embora distintas da tipagem tradicional dos compiladores convencionais, seguem o mesmo princípio geral, isto é, identificar inconsistências estruturais antes que o autômato seja submetido à simulação descrita na Seção 2.7.

3.5 AST (ÁRVORE SINTÁTICA ABSTRATA)

A árvore de derivação produzida pela análise sintática, também chamada de árvore sintática concreta, espelha fielmente a aplicação das produções da gramática durante o reconhecimento da entrada. Por essa razão, ela costuma conter nós internos cuja função é puramente auxiliar, isto é, nós que existem apenas para resolver questões gramaticais como precedência, associatividade ou eliminação de recursão à esquerda, mas que não correspondem a nenhuma construção semanticamente significativa da

linguagem fonte (Aho *et al.*, 2008). Para as fases posteriores do compilador, esses nós auxiliares representam ruído, pois aumentam o tamanho da árvore sem agregar informação útil sobre o programa.

A árvore sintática abstrata (*abstract syntax tree*, ou AST) é uma representação intermediária que elimina esse ruído, conservando apenas a estrutura essencial do programa fonte. Em uma AST de uma expressão, cada nó interno representa um operador ou uma construção da linguagem, e seus filhos representam os operandos ou componentes semanticamente significativos dessa construção (Aho *et al.*, 2008). Não há nós para variáveis sintáticas auxiliares nem para produções unitárias, e os terminais que servem apenas como pontuação (parênteses, ponto-e-vírgula, palavras-chave de delimitação) costumam ser descartados, pois sua função já está implícita na estrutura hierárquica da árvore. O resultado é uma estrutura mais compacta, mais próxima da intenção do programador e, por consequência, mais conveniente para ser percorrida pelas fases de análise semântica, otimização e geração de código.

A Figura 8 apresenta a AST da expressão $id + id * id$. Comparada à árvore de derivação da mesma expressão, exibida na Figura 7, observa-se a redução significativa do número de nós, pois desaparecem todos os nós internos rotulados com as variáveis auxiliares E, T e F, preservando-se apenas os operadores e os operandos efetivos. A precedência dos operadores, antes expressa pela profundidade dos nós auxiliares, agora é capturada diretamente pela própria estrutura da árvore.

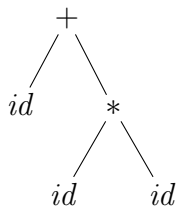


FIGURA 8 – ÁRVORE SINTÁTICA ABSTRATA DA EXPRESSÃO $id + id * id$

FONTE: Os autores (2026)

A construção da AST costuma ser entrelaçada com a análise sintática, por meio das ações semânticas associadas a cada produção da gramática, conforme o esquema da tradução dirigida por sintaxe descrito na Seção 3.3. À medida que o analisador sintático reconhece uma produção, executa-se a ação correspondente, que tipicamente cria um nó da AST e o conecta aos nós já criados para os componentes da produção. Ao término da análise, o ponteiro para a raiz da árvore representa toda a estrutura do

programa fonte e é repassado às fases seguintes (Aho *et al.*, 2008).

Do ponto de vista de implementação, cada nó de uma AST é, em geral, um registro contendo um rótulo que identifica o tipo do nó (por exemplo, atribuição, soma, identificador, literal), um conjunto de ponteiros para os filhos e, eventualmente, atributos específicos do tipo de nó, como o nome de um identificador ou o valor de uma constante (Aho *et al.*, 2008). As fases subsequentes do compilador percorrem essa estrutura, lendo e anotando os nós conforme suas necessidades. A análise semântica, por exemplo, anota cada nó com seu tipo inferido e, quando necessário, insere nós adicionais representando conversões implícitas. O gerador de código intermediário, por sua vez, percorre a árvore anotada e emite, para cada nó, as instruções correspondentes na linguagem intermediária.

Para os fins deste trabalho, a AST representa o objeto central manipulado pela ferramenta proposta. Cada programa escrito na linguagem de domínio específico descreve um autômato finito, e sua AST organiza, em uma estrutura hierárquica única, os componentes desse autômato, ou seja, o conjunto de estados, o alfabeto, o conjunto de transições, o estado inicial e o conjunto de estados finais. É sobre essa estrutura, e não sobre o texto fonte, que operam tanto a análise semântica, descrita na Seção 3.4, quanto a etapa subsequente de simulação visual, conforme apresentada na Seção 2.7.

3.6 LINGUAGENS DE DOMÍNIO ESPECÍFICO (DSL)

As seções anteriores apresentaram os fundamentos do processo de compilação tomando como cenário implícito o de uma linguagem de programação de propósito geral, isto é, uma linguagem projetada para expressar qualquer programa computável, como C, Java ou Rust. Existe, contudo, uma classe distinta de linguagens, voltadas não para a generalidade, mas para a expressividade restrita a um domínio particular. Essa categoria recebe o nome de linguagem de domínio específico ou, na sigla consagrada em inglês, *Domain-Specific Language* (DSL). Esta seção apresenta o conceito de DSL, suas principais variantes e a relação que mantém com o ferramental de compiladores discutido nas seções precedentes.

Fowler e Parsons (2010) definem uma DSL como uma linguagem de programação de expressividade limitada, voltada para um domínio específico. A definição é deliberadamente curta, mas seus quatro elementos centrais merecem atenção. Em pri-

meiro lugar, uma DSL é uma *linguagem de programação*, no sentido de que se destina a ser escrita e lida por seres humanos para expressar instruções a serem processadas por um computador, e não meramente um formato de dados. Em segundo lugar, é necessária a existência de uma *natureza linguística*, ou seja, os fragmentos isolados da DSL devem compor-se em construções maiores cujo sentido depende da forma como se articulam, e não apenas da soma das partes. Em terceiro lugar, sua *expressividade é limitada*, no sentido de que oferece apenas os recursos estritamente necessários para descrever o domínio que pretende cobrir, sem aspirar à generalidade computacional. Por fim, é *focada em um domínio*, e essa restrição de escopo é justamente o que torna a linguagem útil, pois permite que tanto a sintaxe quanto a semântica sejam moldadas em torno dos conceitos naturais daquele domínio.

A consequência prática dessa restrição é que uma DSL bem projetada permite que problemas do seu domínio sejam expressos de maneira mais concisa, mais legível e mais próxima da terminologia utilizada pelos especialistas do que seria possível em uma linguagem de propósito geral (Fowler; Parsons, 2010). Em contrapartida, uma DSL não se presta à escrita de programas fora do seu escopo. Não se trata de uma deficiência, mas de uma decisão de projeto, pois a perda de generalidade é o preço pago para obter um vocabulário e uma sintaxe mais alinhados ao problema que se quer resolver.

Fowler e Parsons (2010) distinguem duas grandes categorias de DSLs conforme a estratégia de implementação adotada. Uma DSL externa é uma linguagem com sintaxe própria, independente da sintaxe de qualquer linguagem hospedeira, e cujo processamento exige a construção de um analisador, em geral por meio das técnicas clássicas de análise léxica e sintática descritas nas Seções 3.2 e 3.3. Exemplos consagrados desse tipo incluem expressões regulares, SQL, a linguagem de descrição de gráficos do Graphviz e os formatos de configuração estruturada com regras próprias. Uma DSL interna, por sua vez, é construída sobre uma linguagem hospedeira de propósito geral, aproveitando sua sintaxe e seu sistema de tipos para exprimir conceitos do domínio na forma de uma biblioteca, com nomes de funções e composições sintáticas escolhidos de modo a parecer uma linguagem própria. Linguagens como Ruby, Lisp, Scala e Kotlin são particularmente populares como anfitriãs de DSLs internas, devido à flexibilidade sintática que oferecem.

Cada uma das duas estratégias apresenta vantagens distintas. A DSL interna tem custo de implementação reduzido, pois reaproveita o compilador, o sistema de tipos e o ecossistema de ferramentas da linguagem hospedeira. Em compensação, sua sintaxe está limitada por aquilo que a hospedeira permite expressar, e o usuário do domínio acaba exposto a detalhes da linguagem subjacente. A DSL externa tem o custo inicial mais alto, uma vez que demanda a construção de um *front-end* dedicado, mas oferece liberdade total na escolha da sintaxe e isolamento completo em relação a qualquer linguagem hospedeira, o que costuma resultar em uma notação mais próxima do vocabulário do domínio (Fowler; Parsons, 2010).

Independentemente da categoria, Fowler e Parsons (2010) argumentam que toda DSL bem estruturada se organiza em torno de dois elementos. O primeiro é a representação textual, isto é, a sintaxe concreta que o usuário efetivamente lê e escreve. O segundo é o modelo semântico, uma estrutura de objetos, em geral construída em uma linguagem de propósito geral, que captura o significado do que foi escrito e sobre a qual se efetivamente operam as funcionalidades do sistema. O processamento de um programa escrito na DSL consiste, então, em traduzir a representação textual no correspondente modelo semântico, tarefa para a qual o ferramental de compiladores apresentado nas seções anteriores se aplica diretamente: a análise léxica converte caracteres em *tokens*, a análise sintática organiza os *tokens* em uma árvore, e fases subsequentes de tradução dirigida pela sintaxe constroem o modelo semântico a partir dessa árvore.

Essa visão é particularmente útil para situar o presente trabalho. A ferramenta proposta adota uma DSL externa, com sintaxe própria, voltada exclusivamente para a descrição declarativa de autômatos finitos. Seu modelo semântico é a representação interna do autômato, isto é, o conjunto de estados, o alfabeto, o conjunto de transições, o estado inicial e o conjunto de estados finais, organizados em uma estrutura de dados sobre a qual operam o verificador semântico e o simulador visual. As fases descritas nas Seções 3.2 a 3.4 correspondem precisamente ao processo de tradução da forma textual para esse modelo, e a árvore sintática abstrata, apresentada na Seção 3.5, é a estrutura intermediária por meio da qual essa tradução se concretiza.

3.7 CONSTRUÇÃO DE COMPILADORES (FERRAMENTAS)

A descrição apresentada nas seções anteriores dá conta de *o que* cada fase de um compilador realiza, mas é silenciosa sobre *como* essas fases são efetivamente implementadas em um sistema concreto. Construir manualmente, do zero, um analisador léxico, um analisador sintático e os mecanismos de tradução dirigida pela sintaxe é um esforço considerável e propenso a erros, sobretudo no caso de uma DSL externa, em que toda a infraestrutura precisa ser provida pelo próprio implementador. Por essa razão, ao longo das décadas consolidou-se um conjunto de ferramentas e bibliotecas voltadas justamente a automatizar partes substanciais desse trabalho. Esta seção apresenta as principais classes de ferramentas utilizadas na construção de compiladores, em particular daqueles destinados a DSLs, organizadas conforme a classificação proposta por Fowler e Parsons (2010).

A primeira e mais tradicional classe de ferramentas é a dos geradores de analisadores sintáticos, também chamados de *parser generators*. Nessa abordagem, o implementador descreve a linguagem por meio de uma gramática livre de contexto, escrita em uma notação próxima da BNF apresentada na Seção 3.3, eventualmente acrescida de ações semânticas associadas a cada produção. A ferramenta toma essa especificação como entrada e produz, automaticamente, o código fonte de um analisador completo, em geral em uma linguagem de propósito geral como C, Java ou Python (Fowler; Parsons, 2010). Pertencem a essa família ferramentas como Yacc e Bison, voltadas à análise ascendente do tipo LR, e ANTLR, particularmente popular para análise descendente preditiva. A vantagem central dessa abordagem é a correspondência direta entre a descrição formal da linguagem e o analisador gerado, o que torna o resultado fácil de revisar e de modificar quando a linguagem evolui. Em contrapartida, o uso desse tipo de ferramenta exige certa familiaridade com a teoria de gramáticas livres de contexto e com as limitações específicas da classe de *parsers* que a ferramenta produz, como as restrições LR, LL ou LALR.

Uma segunda classe de ferramentas é a dos combinadores de *parsers*, ou *parser combinators*. Em vez de gerar código a partir de uma gramática externa, essa abordagem fornece, na forma de uma biblioteca dentro de uma linguagem hospedeira, um conjunto de *parsers* elementares, capazes de reconhecer fragmentos triviais como um único caractere, uma palavra-chave ou um número, junto a um conjunto de combinado-

res que permitem construir *parsers* mais complexos a partir desses elementos por meio de composição (Fowler; Parsons, 2010). A gramática da linguagem aparece, então, no próprio código fonte do compilador, expressa diretamente pelas funções e operadores da biblioteca. Essa estratégia é particularmente comum em linguagens funcionais, como Haskell e Scala, mas também encontra expressão em linguagens modernas com bom suporte a tipos algébricos, sendo amplamente utilizada no ecossistema Rust por meio de bibliotecas como nom e chumsky. Sua principal vantagem é a ausência de qualquer etapa adicional de geração de código, o que integra perfeitamente o analisador ao restante do compilador, em uma única linguagem e em um único processo de compilação. A contrapartida é que o desempenho e a clareza dos diagnósticos podem depender de forma sensível da maneira como a gramática é estruturada, exigindo do implementador atenção a questões como retrocesso e ambiguidade.

Uma terceira opção, frequentemente subestimada, é a do analisador escrito manualmente, em geral pela técnica de descida recursiva, descrita na Seção 3.3. Nessa abordagem, cada não-terminal da gramática é traduzido em uma função do compilador, que invoca, direta ou indiretamente, as demais funções correspondentes aos não-terminais que aparecem em suas produções. Apesar de demandar mais trabalho inicial do que as duas alternativas anteriores, essa estratégia oferece o controle mais fino sobre o comportamento do analisador, sendo especialmente útil quando se deseja produzir mensagens de erro de alta qualidade ou quando a linguagem inclui construções que se acomodam mal às restrições de um gerador automático (Fowler; Parsons, 2010). Não por acaso, muitos compiladores de produção, incluindo os de linguagens de propósito geral amplamente utilizadas, optam pela escrita manual do analisador.

Uma quarta classe, mais recente, é a das *language workbenches*, ou plataformas para engenharia de linguagens. Diferentemente das ferramentas anteriores, que se concentram quase exclusivamente na construção do *front-end*, uma *language workbench* oferece suporte integrado a todas as etapas de definição de uma DSL, da gramática à semântica, passando pelas ferramentas de edição, de verificação e até de geração de código (Fowler; Parsons, 2010). Sistemas como Xtext, JetBrains MPS e Spoofax permitem que o desenvolvedor da linguagem defina, em um único ambiente, a sintaxe, o sistema de tipos, os transformadores e os geradores, recebendo

em troca um editor com realce de sintaxe, completamento automático e diagnóstico em tempo real, tudo derivado da própria definição da linguagem. Essa abordagem amplia consideravelmente a produtividade do projetista de DSLs, mas implica adesão a um ecossistema específico e introduz uma curva de aprendizado nada trivial.

A escolha entre essas alternativas depende de fatores como a complexidade da linguagem, o público-alvo, as exigências de desempenho do analisador e a familiaridade da equipe com as diferentes técnicas (Fowler; Parsons, 2010). Para os fins do presente trabalho, em que a DSL proposta é compacta, voltada exclusivamente à descrição de autômatos finitos, e cujo compilador integra um sistema mais amplo escrito em uma única linguagem hospedeira, a estratégia adotada combina três recursos: um gerador de analisadores léxicos embutido na própria linguagem hospedeira, que produz o *lexer* a partir de anotações no código-fonte; uma biblioteca de *parser combinators* para a análise sintática; e a escrita manual da análise semântica e das demais etapas. Essa combinação preserva a integração natural com o restante da ferramenta, tal como apresentada no Capítulo 4, sem introduzir etapas externas de geração de código nem dependências de plataformas de *language engineering*.

3.8 DIAGNÓSTICO DE ERROS

A capacidade de relatar erros de forma precisa e útil é, ao lado da própria tradução, uma das duas funções essenciais de um compilador, conforme observado já na Seção 3.1. Em compiladores de linguagens de propósito geral, essa exigência costuma ser tratada como uma preocupação de engenharia entre outras, mas em compiladores de DSLs ela assume um peso ainda maior, pois o público que escreve programas em uma DSL inclui frequentemente especialistas do domínio sem formação extensa em programação (Fowler; Parsons, 2010). Para esses usuários, uma mensagem de erro confusa, fora de posição ou expressa em vocabulário técnico estranho ao domínio pode inviabilizar o uso da linguagem com a mesma eficácia de um defeito de tradução. O presente trabalho, voltado a estudantes de teoria da computação e dirigido também a usuários em fase de aprendizagem, enquadra-se justamente nesse cenário, e por isso o diagnóstico de erros recebe aqui tratamento próprio.

Os erros que um compilador pode detectar distribuem-se ao longo das três fases do *front-end*, conforme já exposto nas seções correspondentes. A análise léxica detecta

erros de formação de *tokens*, como caracteres isolados que não pertencem ao alfabeto da linguagem ou literais malformados. A análise sintática detecta erros de estrutura, isto é, sequências de *tokens* que, embora individualmente válidas, não compõem uma derivação admitida pela gramática, como a ausência de um delimitador esperado ou o uso de uma construção fora de contexto. A análise semântica detecta, por fim, erros que escapam à descrição puramente sintática, tais como o uso de identificadores não declarados, incompatibilidades de tipo ou violações de restrições próprias do domínio que a linguagem modela. Cada fase, portanto, contribui com um vocabulário próprio de erros, e cada uma demanda mecanismos específicos para detectá-los e para recuperar-se deles a fim de prosseguir o processamento.

Independentemente da fase em que ocorrem, a utilidade de um erro depende diretamente da qualidade com que é relatado. Aho *et al.* (2008) estabelecem como objetivos do tratamento de erros em um compilador relatar a presença de erros de forma clara e precisa, recuperar-se de cada erro com rapidez suficiente para detectar os erros subsequentes e acrescentar sobrecarga mínima ao processamento de programas corretos. A partir desses objetivos, e considerando o público em fase de aprendizagem a que se dirige a ferramenta proposta, adotam-se neste trabalho três critérios para avaliar a qualidade de um diagnóstico. O primeiro é a precisão da localização, isto é, a indicação inequívoca, em termos de linha e coluna do arquivo fonte, do ponto do programa em que o erro foi detectado. O segundo é a clareza da mensagem, ou seja, a expressão do erro em uma linguagem natural compreensível para o usuário, evitando jargão técnico desnecessário e, sempre que possível, indicando o que se esperava encontrar e o que foi de fato encontrado. O terceiro é a capacidade de detectar múltiplos erros em uma única execução, evitando que o usuário tenha de repetir o ciclo de compilação diversas vezes para descobrir, sucessivamente, defeitos que poderiam ter sido reportados em conjunto.

A recuperação após o primeiro erro, requisito para que o compilador prossiga a análise e detecte os erros remanescentes, depende das estratégias já discutidas na Seção 3.3. Para erros sintáticos, o modo pânico oferece uma recuperação simples e robusta, ao custo de ignorar trechos do programa fonte; a recuperação em nível de frase tenta inserir, remover ou substituir *tokens* para reconstruir uma forma sintaticamente válida, com riscos controlados de provocar erros em cascata; e as produções de erro

permitem que o projetista da linguagem antecipe construções incorretas frequentes e atribua a elas mensagens específicas. Para erros semânticos, a recuperação é, em geral, mais simples, pois a estrutura do programa já foi reconhecida pela fase sintática, restando apenas anotar o erro detectado e prosseguir a verificação dos demais nós da árvore.

No contexto de DSLs, Fowler e Parsons (2010) observam que a qualidade do diagnóstico está intimamente ligada à escolha da estratégia de implementação discutida na Seção 3.7. Geradores automáticos de *parsers* costumam produzir mensagens genéricas, expressas em termos de *tokens* e estados internos do analisador, que pouco ajudam o usuário final. Combinadores de *parsers* permitem maior controle sobre as mensagens, mas requerem atenção do implementador para que esse controle seja efetivamente exercido. Analisadores escritos manualmente, por sua vez, oferecem o mais alto grau de personalização das mensagens, sendo essa, conforme já apontado, uma das principais razões pelas quais compiladores de produção continuam a ser escritos dessa maneira. *Language workbenches* costumam ir além do diagnóstico em *batch* e oferecer detecção de erros incremental, em tempo de edição, diretamente na interface gráfica do editor.

Para a ferramenta proposta neste trabalho, o diagnóstico de erros é tratado como parte central do projeto da DSL, e não como decoração posterior. Cada fase do compilador, da análise léxica à análise semântica, é instrumentada para registrar os erros encontrados em uma estrutura comum, preservando a localização exata no texto fonte e uma mensagem redigida em vocabulário próximo ao domínio dos autômatos finitos, com referências aos conceitos do Capítulo 2, como estados, transições, alfabeto e estados finais. Após o término de cada fase, esses erros são apresentados em conjunto ao usuário, e somente quando uma fase produz erros que impeçam a continuação do processamento é que o restante do compilador deixa de ser executado. Esse tratamento dá ao usuário uma visão panorâmica dos defeitos do programa em cada execução, em linha com os critérios de qualidade enumerados anteriormente, e prepara o terreno para os recursos visuais de apoio descritos no Capítulo 7.

3.9 DSL NO ENSINO

As seções anteriores deste capítulo apresentaram as DSLs como ferramentas voltadas a tornar mais concisa e expressiva a descrição de soluções em um domínio particular. Quando esse domínio é constituído pelos próprios objetos de estudo de uma disciplina acadêmica, a DSL passa a desempenhar, além do papel técnico habitual, uma função pedagógica, isto é, a de servir como linguagem por meio da qual o estudante exerce, formaliza e verifica os conceitos que aprende. Esta seção examina, com base no trabalho de Morazán e Antunez (2014), o uso de DSLs no ensino de linguagens formais e teoria da computação, situando essa prática como contexto imediato do presente trabalho.

O ensino tradicional dos conteúdos discutidos no Capítulo 2 costuma apoiar-se em ferramentas gráficas que permitem ao estudante construir autômatos e gramáticas manipulando diretamente seus elementos visuais, dos quais o exemplo mais difundido é o JFLAP (Rodger; Finley, 2006). Esse tipo de abordagem oferece, sem dúvida, ganhos importantes, em especial a redução do esforço inicial necessário para que o estudante visualize um autômato e acompanhe sua execução. No entanto, conforme apontam Morazán e Antunez (2014), a ênfase exclusiva em ferramentas gráficas tende a separar o estudo de linguagens formais do restante das atividades de programação que o estudante realiza ao longo do curso, tratando o autômato como um diagrama a ser desenhado, e não como um objeto computacional a ser construído e manipulado por meio de código.

A alternativa proposta por Morazán e Antunez (2014) consiste em oferecer ao estudante uma DSL textual em que autômatos finitos, autômatos de pilha, máquinas de Turing e gramáticas sejam descritos diretamente como termos da linguagem, podendo então ser construídos, compostos, testados e executados como qualquer outro programa. Os autores argumentam que essa abordagem recoloca os objetos da teoria da computação no mesmo plano dos demais artefatos com que o estudante trabalha em programação, integrando o estudo das linguagens formais ao restante do currículo de computação em vez de tratá-lo como uma disciplina insulada com ferramental próprio.

Entre os ganhos pedagógicos dessa abordagem, os autores destacam a possibilidade de o estudante escrever, ele mesmo, programas que constroem autômatos a partir de outras estruturas, que verificam propriedades de máquinas dadas ou que com-

param computações em diferentes formalismos, exercícios praticamente inviáveis em um ambiente exclusivamente gráfico (Morazán; Antunez, 2014). Outro ganho relevante é a possibilidade de submeter os autômatos descritos na DSL a testes automatizados, no estilo dos testes unitários usuais em desenvolvimento de software, o que introduz, já no contexto da disciplina de linguagens formais, hábitos de verificação que serão úteis ao estudante em todo o restante de sua formação.

A abordagem textual não se opõe, entretanto, à visualização. Pelo contrário, uma DSL voltada ao ensino tende a beneficiar-se da combinação entre uma representação textual primária, em que o estudante exprime suas ideias, e uma representação visual derivada, gerada automaticamente a partir do texto, que serve para a inspeção do resultado e para a execução passo a passo da máquina descrita. Esse arranjo preserva os ganhos de programação enumerados anteriormente sem abrir mão dos ganhos visuais que motivam o uso de ferramentas como o JFLAP, e é precisamente esse o equilíbrio que a ferramenta proposta no presente trabalho busca alcançar, conforme detalhado nos capítulos subsequentes.

4 RUST COMO LINGUAGEM DE IMPLEMENTAÇÃO

Este capítulo apresenta a linguagem de programação Rust e justifica sua adoção como plataforma de implementação da ferramenta proposta. Inicia-se com uma visão geral da linguagem e de suas características centrais (Seção 4.1), seguindo-se uma discussão sobre seu sistema de tipos e o modelo de propriedade de memória (Seção 4.2). Em seguida, apresenta-se o ecossistema de bibliotecas e o gerenciador de pacotes Cargo (Seção 4.3), e discutem-se as principais bibliotecas de *parser combinators* disponíveis para Rust (Seção 4.4), cujas características orientam a escolha de implementação descrita no Capítulo 7.

4.1 VISÃO GERAL DA LINGUAGEM

Rust é uma linguagem de programação de sistemas criada pela Mozilla Research e lançada em sua primeira versão estável em 2015 (Klabnik; Nichols, 2019). Seu projeto parte de uma premissa central: oferecer desempenho comparável ao de C e C++, sem abrir mão das garantias de segurança de memória que essas linguagens historicamente não fornecem em tempo de compilação. Para atingir esse objetivo, Rust introduz um sistema de tipos estático e um conjunto de regras de compilação, coletivamente denominado modelo de *ownership* e *borrowing*, que permite ao compilador verificar, antes da execução, a ausência de erros como acessos a regiões de memória inválidas, condições de corrida e uso de ponteiros nulos (Klabnik; Nichols, 2019).

A linguagem é compilada para código nativo por meio do compilador `rustc`, que utiliza como *backend* a infraestrutura LLVM, a mesma empregada por compiladores como Clang e Swift (Klabnik; Nichols, 2019). O resultado é um binário executável diretamente pelo sistema operacional, sem a necessidade de uma máquina virtual ou de um interpretador em tempo de execução. Essa característica torna Rust adequado tanto para o desenvolvimento de sistemas de baixo nível, como kernels e drivers, quanto para aplicações de alto nível que exigem desempenho previsível, como compiladores, servidores e ferramentas de linha de comando.

Do ponto de vista da expressividade, Rust combina paradigmas de programação imperativa, funcional e orientada a objetos por meio de *traits*, que funcionam de forma análoga às interfaces de Java ou às classes de tipos de Haskell (Klabnik; Nichols,

2019). Essa flexibilidade permite que construções comuns em linguagens funcionais, como mapeamento, filtragem e encadeamento de iteradores, sejam expressas de forma concisa e com custo de abstração zero, isto é, sem sobrecarga de desempenho em relação ao código imperativo equivalente. Para a construção de compiladores e processadores de linguagens, esse conjunto de características é particularmente valioso: a segurança de tipos facilita a representação de estruturas como *tokens*, nós de AST e estados de autômatos sem o risco de inconsistências em tempo de execução, enquanto o modelo funcional favorece a composição das fases de análise de forma clara e modular.

4.2 SISTEMA DE TIPOS, *OWNERSHIP* E *BORROWING*

O traço mais distintivo de Rust em relação a outras linguagens de sistemas é seu modelo de gerenciamento de memória, que elimina a necessidade de um coletor de lixo (*garbage collector*) sem exigir que o programador gerencie a memória manualmente (Klabnik; Nichols, 2019). Esse modelo é fundamentado em três conceitos inter-relacionados: *ownership* (propriedade), *borrowing* (empréstimo) e *lifetimes* (tempos de vida).

O princípio de *ownership* estabelece que cada valor em Rust possui exatamente um dono em qualquer ponto do programa, e que o valor é automaticamente liberado da memória quando seu dono sai de escopo (Klabnik; Nichols, 2019). A atribuição de um valor a uma nova variável, ou sua passagem como argumento a uma função, transfere a propriedade para o novo contexto, invalidando o acesso anterior. Essa regra, verificada estaticamente pelo compilador, garante que não existam dois fragmentos de código tentando liberar a mesma região de memória simultaneamente.

O *borrowing* complementa o *ownership* ao permitir que um valor seja referenciado sem que sua propriedade seja transferida. Rust distingue dois tipos de referências: as referências imutáveis (`&T`), das quais podem existir simultaneamente quantas forem necessárias, e as referências mutáveis (`&mut T`), das quais apenas uma pode existir em qualquer instante, excluindo a coexistência com qualquer referência imutável (Klabnik; Nichols, 2019). Essas restrições, verificadas pelo compilador em uma fase denominada *borrow checking*, eliminam em tempo de compilação toda uma classe de erros relacionados à mutação concorrente de dados compartilhados.

O sistema de tipos de Rust é estático e fortemente tipado, com suporte a inferência de tipos que dispensa, na maior parte dos casos, anotações explícitas de tipo (Klabnik; Nichols, 2019). Para a representação de estruturas de dados que podem assumir diferentes formas, Rust oferece os tipos enumerados (`enum`), que permitem associar dados distintos a cada variante e são processados por meio de correspondência de padrões (*pattern matching*) com verificação de exaustividade pelo compilador. Esse recurso é particularmente útil na implementação de compiladores, onde nós de AST, *tokens* e erros costumam ser representados naturalmente como enumerações: o compilador garante que todo código que manipula essas estruturas trate explicitamente cada variante possível, evitando casos não tratados silenciosos.

No contexto do presente trabalho, o sistema de tipos e o modelo de *ownership* de Rust oferecem garantias concretas para a implementação das fases do compilador da DSL proposta. A AST do autômato, por exemplo, pode ser representada como uma enumeração cujas variantes correspondem aos diferentes nós da árvore, estados, transições, declarações de alfabeto e metadados, com a certeza de que qualquer acesso a essas estruturas será verificado em tempo de compilação. Da mesma forma, a passagem da AST entre as fases de análise semântica e de simulação não requer cópias desnecessárias, pois o modelo de *borrowing* permite que cada fase leia a estrutura sem assumir sua propriedade.

4.3 ECOSSISTEMA E GERENCIADOR DE PACOTES CARGO

O ecossistema de Rust organiza-se em torno do Cargo, ferramenta oficial que acumula as funções de gerenciador de pacotes, sistema de *build* e executor de testes (Klabnik; Nichols, 2019). Por meio do Cargo, o desenvolvedor declara as dependências do projeto em um arquivo de manifesto (`Cargo.toml`), e a ferramenta resolve automaticamente as versões compatíveis, baixa os pacotes do repositório central `crates.io` e compila todo o projeto em uma única invocação. Esse modelo elimina a necessidade de sistemas de *build* externos e garante que projetos Rust sejam reprodutíveis em qualquer ambiente que disponha do compilador instalado.

A unidade de distribuição de código em Rust é denominada *crate*, termo que corresponde, aproximadamente, ao conceito de biblioteca ou módulo em outras linguagens (Klabnik; Nichols, 2019). O repositório `crates.io` hospeda dezenas de

milhares de *crates* de código aberto, cobrindo desde estruturas de dados e algoritmos até bibliotecas para desenvolvimento *web*, interfaces gráficas e, em especial para os fins deste trabalho, análise sintática e processamento de linguagens.

A integração entre Cargo e o compilador `rustc` estende-se à execução automatizada de testes unitários e de integração. Todo projeto Rust pode incluir módulos de teste diretamente no código fonte, anotados com o atributo `#[test]`, que são compilados e executados pelo comando `cargo test` (Klabnik; Nichols, 2019). Esse mecanismo nativo de testes é relevante para o presente trabalho, pois permite validar cada fase do compilador da DSL de forma isolada, verificando o comportamento do analisador léxico, do analisador sintático e do verificador semântico sobre entradas controladas, em linha com a estratégia de validação descrita no Capítulo 8.

4.4 *PARSER COMBINATORS* EM RUST

Conforme discutido na Seção 3.7, a estratégia de implementação adotada para o compilador da DSL proposta baseia-se no uso de uma biblioteca de *parser combinators*, abordagem que integra naturalmente a definição da gramática ao restante do código da ferramenta. O ecossistema Rust oferece três bibliotecas principais para esse fim: nom, pest e chumsky, cada uma com características distintas que as tornam adequadas a diferentes cenários.

A biblioteca nom é uma das mais antigas e consolidadas do ecossistema Rust para análise sintática (Couprie, 2015). Sua abordagem é baseada em funções que recebem uma fatia de bytes ou caracteres e retornam o resultado do reconhecimento juntamente com o restante da entrada não consumida, compondo-se por meio de macros e funções de ordem superior. A orientação de nom para o processamento de formatos binários e protocolos de rede, domínio em que a eficiência é crítica, torna-a uma escolha robusta para linguagens textuais, embora a geração de mensagens de erro precisas requeira esforço adicional do implementador, pois a biblioteca não foi projetada com esse objetivo em primeiro plano.

A biblioteca pest adota uma abordagem diferente: em vez de compor *parsers* no código Rust, o desenvolvedor descreve a gramática da linguagem em um arquivo externo com notação própria, derivada de gramáticas de expressão de análise (*Parsing Expression Grammars*, ou PEG), e a biblioteca gera automaticamente o analisador

correspondente em tempo de compilação por meio de macros procedurais (Tiselice, 2018). Essa estratégia separa claramente a especificação da gramática de sua implementação, facilitando a leitura e a manutenção da definição da linguagem, mas reduz o controle sobre o comportamento do analisador e sobre as mensagens de erro produzidas.

A biblioteca chumsky é a mais recente das três e foi projetada explicitamente com foco na qualidade do diagnóstico de erros (Barretto, 2022). Seus combinadores propagam automaticamente informações de localização ao longo do processo de análise e oferecem mecanismos nativos de recuperação de erros, permitindo que o analisador continue após encontrar uma construção inválida e reporte múltiplos erros em uma única execução. Essas características alinham-se diretamente com os requisitos de diagnóstico estabelecidos na Seção 3.8, tornando chumsky uma alternativa particularmente adequada para DSLs voltadas ao ensino, em que a qualidade das mensagens de erro é tão importante quanto a correção da análise.

O Quadro 3 sintetiza as principais características das três bibliotecas, evidenciando os critérios mais relevantes para a escolha a ser realizada durante a implementação descrita no Capítulo 7.

QUADRO 3 – COMPARAÇÃO ENTRE BIBLIOTECAS DE PARSER COMBINATORS PARA RUST

Critério	nom	pest	chumsky
Abordagem	Combinadores	Gramática PEG externa	Combinadores
Qualidade dos erros	Baixa	Média	Alta
Recuperação de erros	Manual	Limitada	Nativa
Rastreamento de localização	Manual	Automático	Automático
Maturidade	Alta	Alta	Média
Foco original	Dados binários	Linguagens textuais	Linguagens textuais

FONTE: Os autores (2026)

A escolha entre as três bibliotecas foi orientada pelos requisitos estabelecidos para o compilador da DSL: em especial, a necessidade de mensagens de erro com indicação precisa de localização e natureza do problema, conforme descrito na Seção 3.8, e a integração natural com o restante da ferramenta, escrita integralmente em

Rust. A análise comparativa apresentada nesta seção fornece o embasamento para essa decisão, cuja justificativa é apresentada no Capítulo 7.

5 TRABALHOS RELACIONADOS

O desenvolvimento de ferramentas computacionais para apoiar o ensino de autômatos finitos tem uma trajetória longa e relativamente consolidada na literatura de educação em Ciência da Computação. No entanto, conforme se argumenta ao longo deste trabalho, as abordagens existentes compartilham uma limitação comum: a ausência de uma representação textual formal e de um compilador associado capaz de validar e diagnosticar erros com precisão. Este capítulo apresenta e analisa os cinco trabalhos mais relevantes identificados na revisão bibliográfica, organizados conforme sua plataforma e abordagem, evidenciando em cada caso as contribuições e as limitações que motivam o desenvolvimento da ferramenta proposta.

5.1 ANÁLISE LÉXICA EM RUST

O trabalho intitulado *Exploiting Finite State Automata for Efficient Lexical Analysis: A Rust Implementation*, publicado na forma de artigo técnico e tutorial prático por Bruzzone (2023), apresenta uma implementação de analisador léxico em Rust fundamentada diretamente na teoria dos autômatos finitos. O autor parte da constatação de que analisadores léxicos são, em essência, autômatos finitos não-determinísticos aplicados ao reconhecimento de *tokens*, e propõe uma implementação em que cada categoria léxica corresponde a um estado ou conjunto de estados explicitamente codificados na estrutura do autômato.

A relevância deste trabalho para o presente contexto reside justamente nessa ponte explícita entre a teoria dos autômatos, apresentada no Capítulo 2, e sua aplicação prática em uma linguagem de programação moderna. O autor demonstra que as primitivas de Rust, em especial os tipos enumerados e o mecanismo de correspondência de padrões, são particularmente adequados para representar estados e transições de autômatos de forma segura e eficiente, antecipando argumentos desenvolvidos com maior profundidade no Capítulo 4 deste trabalho.

No entanto, o escopo do trabalho de Bruzzone (2023) restringe-se à fase de análise léxica e à sua conexão com a teoria dos autômatos, sem avançar para as demais fases de compilação nem para qualquer forma de simulação visual ou interativa. Trata-se, portanto, de uma contribuição de natureza técnica e didática voltada

a programadores que desejam compreender a implementação de *lexers*, e não de uma ferramenta educacional direcionada ao ensino de autômatos em si. A ferramenta proposta no presente trabalho diferencia-se ao incorporar, além da análise léxica, as fases sintática e semântica completas, a construção da AST e a simulação visual interativa do autômato descrito.

5.2 JFLAP

O JFLAP (*Java Formal Languages and Automata Package*) é uma das ferramentas educacionais mais difundidas para o ensino de linguagens formais e autômatos. Desenvolvida por Susan H. Rodger e colaboradores na Duke University, com origens que remontam ao início da década de 1990, a ferramenta acumula décadas de desenvolvimento contínuo (Rodger; Finley, 2006). O JFLAP permite a criação e simulação de autômatos finitos determinísticos e não-determinísticos, autômatos de pilha, máquinas de Turing, gramáticas livres de contexto e sistemas-L, constituindo um ambiente abrangente para toda a hierarquia de Chomsky (Rodger; Finley, 2006).

A abordagem do JFLAP é exclusivamente gráfica e baseada em manipulação direta: o usuário constrói o autômato posicionando estados com o *mouse* e conectando transições por meio de arestas rotuladas, sem qualquer forma de representação textual da estrutura produzida (Rodger; Finley, 2006). A simulação passo a passo destaca visualmente os estados e as transições percorridos durante o reconhecimento de uma cadeia, o que confere à ferramenta inegável valor pedagógico para a visualização do comportamento dos autômatos. Rodger e Finley (2006) documentam ainda recursos avançados, como a conversão automática de AFN em AFD pela construção de subconjuntos, a minimização de AFDs e a conversão entre gramáticas e autômatos, todos apresentados com animações visuais.

As limitações do JFLAP, já apontadas na introdução deste trabalho, derivam precisamente de sua natureza exclusivamente visual. A ausência de uma representação textual impede o versionamento, a comparação e o processamento automatizado dos autômatos produzidos. A construção por arrastar-e-soltar não fornece ao estudante diagnósticos precisos quando o autômato está incorretamente especificado: a ferramenta pode aceitar estruturas inválidas sem emitir mensagens de erro localizadas, ou simplesmente não permitir certas configurações sem explicar o motivo. Além disso, por

ser implementada em Java e distribuída como aplicação *desktop*, o JFLAP apresenta dependência de ambiente de execução e interface datada, que contrastam com o ecossistema moderno de ferramentas de desenvolvimento ao qual o estudante de Ciência da Computação está habitualmente exposto.

5.3 GAM

O GAM é uma ferramenta educacional desenvolvida na Universidade Federal de Lavras por Jukemura, Nascimento e Uchôa (2005), com o objetivo de apoiar estudantes e professores no estudo da teoria dos autômatos finitos. A ferramenta é implementada em C++ com a biblioteca gráfica GTK/GTKmm e executa em sistemas GNU/Linux e Windows, sendo distribuída livremente para uso acadêmico.

O GAM oferece módulos para a entrada, edição e teste de autômatos finitos determinísticos e não-determinísticos, além de conversões entre formalismos regulares, como a construção de subconjuntos (de AFN para AFD) e a conversão entre autômatos finitos e expressões regulares (Jukemura; Nascimento; Uchôa, 2005). A interface gráfica permite a construção visual dos autômatos e a simulação do processamento de cadeias de entrada, com apresentação parcial das configurações intermediárias percorridas durante o reconhecimento.

As limitações do GAM são de duas ordens. Em primeiro lugar, a ferramenta impõe restrições estruturais significativas: os alfabetos de entrada e saída são limitados a no máximo três símbolos e as máquinas a no máximo dez estados, o que inviabiliza seu uso com autômatos de complexidade moderada, como os utilizados em exemplos clássicos da literatura (Jukemura; Nascimento; Uchôa, 2005). Em segundo lugar, à semelhança do JFLAP, o GAM não oferece nenhuma forma de representação textual dos autômatos produzidos, mantendo a descrição restrita ao domínio visual. A ausência de diagnósticos de erro precisos e localizados, bem como a dependência de uma interface gráfica de construção manual, são limitações partilhadas com as demais ferramentas desta categoria.

5.4 SIMULADOR UFSC

O Simulador UFSC é uma aplicação *web* desenvolvida como Trabalho de Conclusão de Curso na Universidade Federal de Santa Catarina por Nunes, com orientação

de Furtado e Marchi (Nunes, 2017). O trabalho tem como motivação central a constatação de que as ferramentas *desktop* existentes, como o JFLAP, impõem barreiras de instalação e configuração que dificultam sua adoção em ambientes educacionais heterogêneos, e propõe como alternativa um simulador acessível diretamente pelo navegador, sem necessidade de instalação.

A ferramenta suporta autômatos finitos determinísticos e não-determinísticos, autômatos de pilha e máquinas de Turing, permitindo a construção gráfica dos modelos e a simulação visual do processamento de cadeias, com apresentação dos estados e transições percorridos em cada passo (Nunes, 2017). A arquitetura *web* é um diferencial relevante em relação às ferramentas *desktop* analisadas anteriormente, pois elimina dependências de sistema operacional e facilita o compartilhamento de autômatos entre estudantes e professores por meio de URLs.

Contudo, o Simulador UFSC mantém a abordagem exclusivamente gráfica para a construção dos autômatos, sem oferecer qualquer forma de representação textual ou de compilador associado. A ausência de validação formal da estrutura e de diagnósticos de erro precisos é, portanto, uma limitação compartilhada com os demais trabalhos desta categoria. O estudante que constrói um autômato incorreto não recebe indicação clara do ponto e da natureza do problema, cabendo-lhe identificar o erro por inspeção visual, processo que, conforme argumentado por Morazán e Antunez (2014), tende a ser menos eficaz do que o diagnóstico automatizado oferecido por compiladores.

5.5 AUTOMATOGRAPH

O AutomatoGraph é uma ferramenta educacional desenvolvida em Java por Kienetz e Canal (2016), com o objetivo de auxiliar estudantes no estudo de autômatos finitos determinísticos no contexto da disciplina de Linguagens Formais e Autômatos. Em contraste com as ferramentas anteriores, que oferecem suporte a múltiplos tipos de máquinas abstratas, o AutomatoGraph restringe-se deliberadamente aos AFDs, aprofundando o suporte a esse formalismo específico.

A característica mais distintiva do AutomatoGraph em relação às demais ferramentas analisadas é a geração automática da gramática regular correspondente ao autômato construído, recurso que explicita ao estudante a equivalência entre os dois formalismos e reforça a compreensão da hierarquia de Chomsky discutida na

Seção 2.2 (Kienetz; Canal, 2016). A interface apresenta, em colunas paralelas, a lista de passos percorridos pelo autômato durante o reconhecimento de uma cadeia e a gramática regular gerada, permitindo ao estudante observar ambas as representações simultaneamente.

A simulação passo a passo é apenas parcialmente visual: embora a lista de transições percorridas seja exibida textualmente, o diagrama de estados não é animado de forma interativa durante o processamento, o que reduz o impacto pedagógico em comparação com ferramentas que destacam visualmente o estado corrente em tempo real (Kienetz; Canal, 2016). Além disso, o AutomatoGraph mantém a abordagem gráfica para a construção do autômato e não oferece representação textual formal, versionamento ou diagnóstico de erros estruturais. O escopo restrito a AFDs também constitui uma limitação quando se considera o ensino de não-determinismo, ponto em que a ferramenta proposta no presente trabalho, que suporta tanto AFDs quanto AFNs com descrição textual unificada, representa um avanço significativo.

5.6 ANÁLISE COMPARATIVA

A análise dos trabalhos apresentados nas seções anteriores evidencia um padrão comum entre as ferramentas existentes: a grande maioria adota abordagens exclusivamente visuais para a construção de autômatos, o que gera lacunas críticas em termos de validação formal, versionamento e engenharia de software pedagógica. O Quadro 4 sintetiza as características funcionais de cada ferramenta investigada em relação aos critérios técnicos mais relevantes para o presente trabalho. Complementarmente, o Quadro 5 evidencia de que modo cada estudo fundamenta uma decisão de projeto da ferramenta proposta.

QUADRO 4 – COMPARAÇÃO DE RECURSOS TÉCNICOS ENTRE OS TRABALHOS RELACIONADOS

Trabalho	**DSL textual**	**Versionamento**	**Compilador / diagnóstico**	**Simulação visual**	**Simulação passo a passo**	**Voltada ao ensino**	**Plataforma**
Análise Léxica em Rust	Não	Sim	Não	Não	Linha de comando	Não	Rust (Nativo)
JFLAP	Não	Não	Não	Sim	Sim	Sim	Desktop (Java)
GAM	Não	Não	Não	Sim	Parcial	Sim	Desktop (C++/GTK)
Simulador UFSC	Não	Não	Não	Sim	Sim	Sim	Web (Navegador)
AutomatoGraph	Não	Não	Não	Parcial	Parcial	Sim	Desktop (Java)
Este trabalho	**Sim**	**Sim**	**Sim**	**Sim**	**Sim**	**Sim**	**Desktop**

FONTE: Os autores (2026)

Ao analisar criticamente o cenário delineado nos Quadros 4 e 5, observa-se que nenhuma das ferramentas focadas estritamente no ecossistema de ensino (JFLAP, GAM, Simulador UFSC e AutomatoGraph) permite qualquer tipo de edição textual estruturada ou suporte nativo a versionamento de arquivos. O estudante fica restrito a cliques do *mouse*, impossibilitado de utilizar ferramentas de gerência de configuração como o Git.

Além disso, a ausência de retroalimentação imediata se torna evidente na falta completa de uma infraestrutura de compilador nessas soluções didáticas: nenhuma delas possui análise léxica, sintática ou semântica capaz de interceptar erros conceituais na raiz (como transições para estados não declarados) antes da execução. Embora o JFLAP e o Simulador UFSC entreguem uma simulação passo a passo robusta no diagrama de estados, eles tratam o autômato puramente como um desenho animado, isolado da prática de codificação. Por outro lado, abordagens de engenharia pura como o trabalho de Bruzzone (2023) expõem o rigor técnico do Rust e a manipulação via código, mas falham no suporte educacional por carecerem de interfaces gráficas interativas para estudantes iniciantes.

A ferramenta proposta neste trabalho ocupa, portanto, essa lacuna exata de engenharia pedagógica: ela adota o rigor da descrição textual por meio de uma DSL externa dedicada, implementa uma esteira completa de compilador para fornecer

diagnósticos localizados em tempo de compilação e, simultaneamente, renderiza e anima o diagrama de estados inseparável do código fonte, unificando a teoria de linguagens formais e o pragmatismo do desenvolvimento de software em uma única plataforma de experimentação.

QUADRO 5 – CONTRIBUIÇÕES E LACUNAS IDENTIFICADAS NA LITERATURA

Trabalho	**Contribuição / Lacuna identificada para o projeto**
Análise Léxica em Rust	Contribuição: Demonstra a viabilidade prática e a segurança de tipos da linguagem Rust para modelar e codificar estados de autômatos formalmente.
JFLAP	Lacuna: Interface de modelagem puramente visual baseada em cliques; impede o armazenamento estruturado em texto e o versionamento por código.
GAM	Lacuna: Restrição de escopo severa (limite de 10 estados e 3 símbolos), o que inviabiliza o mapeamento de exemplos complexos da literatura.
Simulador UFSC	Contribuição: Apresenta excelente portabilidade via ambiente web, mas perpetua a dependência da construção no domínio puramente gráfico.
AutomatoGraph	Contribuição: Reforça a equivalência formal ao gerar gramáticas automáticas, mas carece de animação interativa do diagrama em tempo real.
Este trabalho	Decisão de projeto: Une o rigor da descrição declarativa textual (Git-friendly) à esteira de compilação pedagógica e à animação visual síncrona.

FONTE: Os autores (2026)

6 PROJETO DA DSL

Este capítulo apresenta o projeto da Linguagem de Domínio Específico (DSL) proposta, descrevendo sua sintaxe concreta, sua gramática formal e as principais decisões de projeto que orientaram seu desenvolvimento. A linguagem, denominada Autosim, foi projetada para permitir a descrição declarativa de autômatos finitos determinísticos (AFDs) e não-determinísticos (AFNs) com transições ε, de forma textual, estruturada e passível de versionamento.

6.1 VISÃO GERAL DA LINGUAGEM

A Autosim é uma DSL externa, isto é, possui sintaxe própria e independente de qualquer linguagem hospedeira, processada por um compilador dedicado implementado em Rust. Sua sintaxe foi projetada em português para alinhar-se ao vocabulário dos estudantes brasileiros de teoria da computação, utilizando termos como `alfabeto`, `automato`, `estados`, `inicial`, `finais`, `transicoes` e `simular`.

Cada arquivo fonte (extensão `.asl`) descreve um alfabeto comum e um ou mais autômatos, seguidos opcionalmente por comandos de simulação que especificam cadeias de entrada a serem testadas. A Listagem 6.1 ilustra um arquivo válido da linguagem.

Listing 6.1 – EXEMPLO DE AFD DESCRITO NA DSL AUTOSIM

```
1  alfabeto { 'a', 'b' }
2
3  automato AFD exemplo_afd {
4      estados { q0, q1, q2 }
5      inicial q0
6      finais { q2 }
7
8      transicoes {
9          q0 -> q1 com 'a'
10         q0 -> q0 com 'b'
11         q1 -> q2 com 'b'
12         q1 -> q1 com 'a'
```

```
13          q2 -> q2 com 'a'
14          q2 -> q2 com 'b'
15      }
16  }
17
18  simular exemplo_afd com "ab"
19  simular exemplo_afd com "aab"
20  simular exemplo_afd com "bbb"
21  simular exemplo_afd com ""
```

O alfabeto é declarado uma única vez no início do arquivo e é compartilhado por todos os autômatos do programa, garantindo consistência entre as declarações. Cada autômato é nomeado e classificado como `AFD` (determinístico) ou `AFN` (não-determinístico), e sua estrutura é composta por quatro seções: estados, estado inicial, estados finais e transições.

6.2 GRAMÁTICA FORMAL

A gramática da linguagem é definida em notação EBNF conforme apresentado na Figura 9. A linguagem é livre de contexto e foi projetada para ser analisável por um *parser* descendente preditivo com um símbolo de *lookahead*.

FIGURA 9 – GRAMÁTICA DA DSL AUTOSIM EM NOTAÇÃO EBNF

```
programa        ::= declaracao_alfabeto item*

declaracao_alfabeto ::= "alfabeto" "{" [ char_literal ("," char_literal)* [","] ] "}"

item            ::= declaracao_automato | comando_simulacao

declaracao_automato ::= "automato" tipo_automato ident "{"
                            "estados" "{" [ ident ("," ident)* [","] ] "}"
                            "inicial" ident
                            "finais" "{" [ ident ("," ident)* [","] ] "}"
                            "transicoes" "{" transicao* "}"
                        "}"

tipo_automato   ::= "AFD" | "AFN"

transicao       ::= ident "->" ident "com" (char_literal | "epsilon" | "eps")

comando_simulacao ::= "simular" ident "com" cadeia_literal

```

FONTE: Os autores (2026)

Os terminais da gramática são definidos conforme as expressões regulares apresentadas no Quadro 6.

QUADRO 6 – TERMINAIS DA DSL AUTOSIM

Terminal	**Padrão**	**Exemplo**
identificador	`[a-zA-Z_][a-zA-Z0-9_]*`	`q0`, `exemplo_afd`
char_literal	`[^']`	`'a'`, `'0'`
cadeia_literal	`[^"]*`	`"ab"`, `""`

FONTE: Os autores (2026)

6.3 PALAVRAS RESERVADAS

A linguagem define onze palavras reservadas, listadas no Quadro 7. Todas as palavras reservadas são insensíveis a maiúsculas e minúsculas (*case-insensitive*), de modo que `AFD`, `afd` e `Afd` são igualmente válidas. Essa decisão visa reduzir a carga cognitiva sobre o estudante, eliminando uma fonte comum de erros em linguagens que distinguem caixa.

QUADRO 7 – PALAVRAS RESERVADAS DA DSL AUTOSIM

Palavra	**Função**
`alfabeto`	Declaração do alfabeto
`automato`	Declaração de autômato
`AFD`	Especificação de autômato determinístico
`AFN`	Especificação de autômato não-determinístico
`estados`	Declaração do conjunto de estados
`inicial`	Definição do estado inicial
`finais`	Declaração dos estados de aceitação
`transicoes`	Declaração das transições
`simular`	Comando de simulação
`com`	Preposição para símbolo de transição
`epsilon` / `eps`	Transição vazia (ε)

FONTE: Os autores (2026)

6.4 DECISÕES DE PROJETO

O projeto da DSL foi orientado por quatro princípios fundamentais, descritos a seguir.

6.4.1 Declaração Única de Alfabeto

Diferentemente de ferramentas como o JFLAP, em que cada autômato pode definir seu próprio alfabeto de forma implícita durante a construção visual, a Autosim exige que o alfabeto seja declarado explicitamente uma única vez no início do arquivo. Essa decisão simplifica a verificação semântica, pois todos os símbolos utilizados em transições podem ser validados contra um único conjunto de referência, e reforça o conceito teórico de que o alfabeto é parte integrante da definição formal do autômato.

6.4.2 Classificação Explícita AFD/AFN

A linguagem exige que o usuário declare explicitamente se o autômato é determinístico (AFD) ou não-determinístico (AFN). Essa classificação permite que o compilador realize verificações específicas para cada tipo: para AFDs, o analisador semântico verifica a ausência de transições ε e a unicidade de transição por par (estado, símbolo); para AFNs, ambas as construções são permitidas. Essa distinção tem valor pedagógico,

pois obriga o estudante a refletir sobre a natureza do autômato que está descrevendo.

6.4.3 Sintaxe em Português

A escolha do português para as palavras reservadas visa reduzir a barreira linguística para estudantes brasileiros, permitindo que o foco permaneça nos conceitos de autômatos e não na memorização de vocabulário em língua estrangeira. Essa decisão está alinhada com o caráter educacional da ferramenta.

6.4.4 Insensibilidade a Maiúsculas e Minúsculas

Todas as palavras reservadas são insensíveis a maiúsculas e minúsculas. Isso elimina erros comuns de digitação e torna a linguagem mais tolerante para estudantes em fase de aprendizado, sem comprometer a clareza da descrição. Os identificadores, por outro lado, diferenciam maiúsculas de minúsculas, de modo que `q0` e `Q0` designam estados distintos, preservando a correspondência exata entre os nomes declarados e os nomes referenciados.

6.5 SUPORTE A TRANSIÇÕES ε

A DSL oferece suporte nativo a transições ε (vazias), características de AFNs. O usuário pode empregar tanto a palavra `epsilon` quanto sua abreviação `eps` como símbolo de transição. Durante a simulação, cada transição ε é percorrida como um passo próprio, que muda o estado sem consumir símbolo da entrada, conforme descrito no Capítulo 7. A Listagem 6.2 ilustra um AFN-ε que reconhece a linguagem a^*b^*.

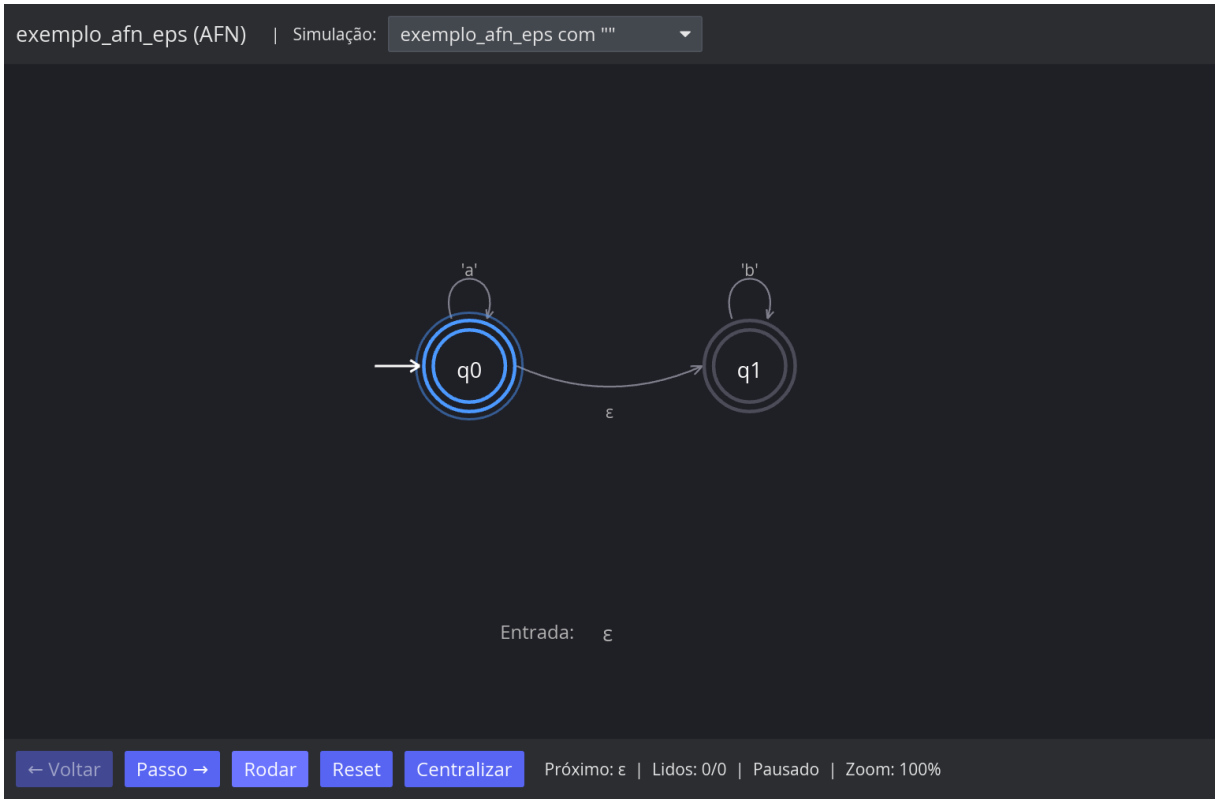
Listing 6.2 – EXEMPLO DE AFN COM TRANSIÇÕES ε

```
1  alfabeto { 'a', 'b' }
2
3  automato AFN exemplo_afn_eps {
4      estados { q0, q1 }
5      inicial q0
6      finais { q0, q1 }
7
8      transicoes {
```

```
9           q0 -> q0 com 'a'
10          q0 -> q1 com eps
11          q1 -> q1 com 'b'
12      }
13  }
14
15  simular exemplo_afn_eps com ""
16  simular exemplo_afn_eps com "aaa"
17  simular exemplo_afn_eps com "aabb"
```

A Figura 10 mostra a interface da ferramenta Autosim renderizando automaticamente o diagrama de estados a partir desse mesmo arquivo fonte, com a transição ε identificada pelo símbolo ε sobre a aresta que liga os estados `q0` e `q1`.

FIGURA 10 – INTERFACE DA FERRAMENTA AUTOSIM RENDERIZANDO O AFN-ε DA LISTAGEM 6.2



FONTE: Os autores (2026)

6.6 TRATAMENTO DE COMENTÁRIOS

A linguagem suporta comentários de linha única iniciados por `//`. Os comentários são descartados durante a análise léxica e não afetam a geração da AST. Não há suporte a comentários multilinha, pois a sintaxe da linguagem é suficientemente compacta para que comentários de linha única sejam adequados.

7 IMPLEMENTAÇÃO

Este capítulo descreve a implementação da ferramenta Autosim, detalhando cada etapa do pipeline de compilação e da interface gráfica interativa. A implementação foi realizada integralmente na linguagem Rust, utilizando as bibliotecas Logos (Hirsz, 2021) para análise léxica, Chumsky (Barretto, 2022) para análise sintática, Iced (Ramón, 2022) para a interface gráfica e Ariadne para o diagnóstico de erros. O código-fonte completo da ferramenta está disponível publicamente em repositório aberto (Vieira; Beckmann, 2026).

7.1 VISÃO GERAL DO PIPELINE

O processamento de um arquivo-fonte `.asl` segue o pipeline ilustrado na Figura 11. Cada fase do pipeline é executada sequencialmente. Em caso de erro em qualquer fase, as mensagens de diagnóstico são exibidas via Ariadne e a execução é interrompida antes da fase seguinte.

FIGURA 11 – PIPELINE DE COMPILAÇÃO DA FERRAMENTA AUTOSIM

Arquivo .asl $\rightarrow$ **Analisador Léxico** (Logos) $\rightarrow$ **Analisador Sintático** (Chumsky) $\rightarrow$ **Analisador Semântico** $\rightarrow$ **Simulador** $\rightarrow$ **Interface Gráfica** (Iced)

FONTE: Os autores (2026)

7.2 ANALISADOR LÉXICO

O analisador léxico é implementado com a biblioteca Logos (Hirsz, 2021), que gera automaticamente o código do *lexer* a partir de anotações declarativas na definição de um tipo enumerado. Cada variante do enumerado `Token` corresponde a uma categoria léxica, anotada com seu padrão de reconhecimento.

7.2.1 Definição dos Tokens

O tipo `Token` é definido com a macro `#[derive(Logos)]`, que gera o autômato finito determinístico para o reconhecimento eficiente dos padrões. As palavras reservadas utilizam o atributo `ignore(case)` para garantir insensibilidade a maiúsculas e minúsculas. São reconhecidas as seguintes categorias de *tokens*:

- Palavras reservadas: `alfabeto`, `automato`, `AFD`, `AFN`, `estados`, `inicial`, `finais`, `transicoes`, `simular`, `com`, `epsilon`, `eps`;
- Símbolos: `ChaveAberta` ({), `ChaveFechada` (}), `Vírgula` (,), `Seta` (`--`);
- Literais: `CharLiteral` (`'a'`), `StringLiteral` (`"ab"`);
- Identificadores: `Ident` (`q0`, `exemplo`).

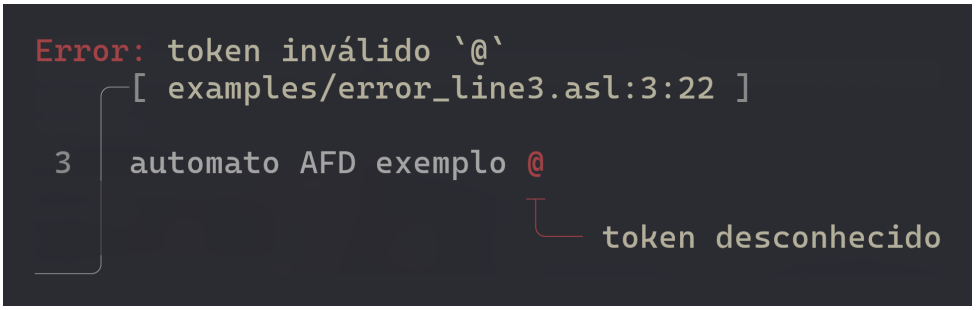
Espaços em branco e comentários (iniciados por `//`) são reconhecidos por padrões específicos e descartados, sem gerar *tokens*.

7.2.2 Tratamento de Erros Léxicos

Quando um caractere não pertence a nenhum padrão definido, o Logos retorna um erro contendo a posição (linha e coluna) do caractere inválido. A ferramenta converte essa informação em uma mensagem de diagnóstico por meio da biblioteca Ariadne, que exibe o trecho do código-fonte com realce colorido, uma seta apontando para a posição do erro e uma mensagem descritiva.

O diagnóstico gerado nessa situação é apresentado na Figura 12, que corresponde ao processamento do arquivo `error_line3.asl`, no qual o caractere `@` ocorre na linha 3, coluna 22. A mensagem identifica o arquivo e a posição exata da ocorrência, reproduz a linha do código-fonte com o caractere inválido em destaque e ancora, logo abaixo dele, o rótulo "token desconhecido", de modo que o estudante localize o problema sem necessidade de inspecionar manualmente o arquivo.

FIGURA 12 – MENSAGEM DE ERRO LÉXICO GERADA PELA BIBLIOTECA ARIADNE



FONTE: Os autores (2026)

7.3 ANALISADOR SINTÁTICO

O analisador sintático é implementado com a biblioteca Chumsky (Barretto, 2022), uma biblioteca de combinadores de *parsers* para Rust. A gramática da DSL é codificada diretamente em código Rust por meio da composição de funções combinadoras, cada uma responsável por reconhecer uma construção específica da linguagem.

A escolha do Chumsky, entre as alternativas comparadas no Quadro 3, decorre dos requisitos de diagnóstico estabelecidos na Seção 3.8. Diferentemente do nom, cuja geração de mensagens de erro precisas exige esforço adicional do implementador, e do pest, que reduz o controle sobre as mensagens ao gerar o analisador a partir de uma gramática externa, o Chumsky foi projetado com foco na qualidade dos diagnósticos: propaga automaticamente a localização dos erros e oferece recuperação nativa, permitindo reportar múltiplos erros em uma única execução. Além disso, por ser uma biblioteca de combinadores, mantém a gramática no próprio código Rust, integrada ao restante da ferramenta, sem etapas externas de geração de código.

7.3.1 Árvore Sintática Abstrata (AST)

A AST da linguagem é definida por um conjunto de estruturas de dados que capturam a essência do autômato descrito, eliminando ruídos sintáticos como delimitadores e palavras-chave. As principais estruturas são:

- `Program`: contém o alfabeto (lista de caracteres), a lista de autômatos e a lista de comandos de simulação;
- `Automaton`: armazena o tipo (AFD ou AFN), o nome, a lista de estados, o estado inicial, a lista de estados finais e a lista de transições;
- `Transition`: representa uma transição com estado de origem, estado de destino e símbolo (caractere do alfabeto ou ε);
- `Simulation`: associa o nome de um autômato a uma cadeia de entrada.

Cada nó da AST preserva sua posição no texto-fonte por meio de um *span* (`Range<usize>`). Como o analisador sintático opera sobre a sequência de *tokens* produzida pelo analisador léxico, e não diretamente sobre os caracteres, esse *span*

registra os índices de início e fim do nó nessa sequência de *tokens*. Na geração das mensagens de erro, tanto sintáticas quanto semânticas, esses índices são convertidos em posições de caractere a partir das posições registradas pelo analisador léxico, o que permite indicar a linha e a coluna exatas do problema.

7.3.2 Estrutura da Gramática

A gramática é implementada como combinadores encadeados. O combinador de mais alto nível, `program()`, reconhece a declaração do alfabeto seguida de zero ou mais itens (declarações de autômato ou comandos de simulação). Cada construção é validada com verificações de presença e ordenação: por exemplo, o corpo de um autômato exige que as quatro seções (`estados`, `inicial`, `finais`, `transicoes`) apareçam obrigatoriamente nessa ordem.

7.3.3 Tratamento de Erros Sintáticos

A Chumsky oferece recuperação automática de erros, permitindo que o analisador continue após encontrar uma construção inválida e acumule múltiplos erros em uma única execução. As mensagens de erro são geradas no formato "esperava X, encontrou Y", com indicação precisa da linha e coluna. Por exemplo, ao encontrar um identificador onde se esperava a palavra `inicial`, a mensagem gerada é: "esperava ’inicial’, encontrou ’q0’".

7.4 ANALISADOR SEMÂNTICO

A análise semântica é realizada em três fases, executadas sequencialmente sobre a AST produzida pelo analisador sintático. Todas as verificações são feitas estaticamente, antes da simulação.

7.4.1 Construção da Tabela de Símbolos

A primeira fase percorre a AST e constrói uma tabela de símbolos contendo o alfabeto e as entradas de cada autômato (estados, inicial, finais). Durante essa fase, são detectados e reportados os seguintes erros:

- Símbolo duplicado no alfabeto;

- Estado duplicado em um mesmo autômato;
- Nome de autômato duplicado.

7.4.2 Verificação de Referências

A segunda fase percorre as transições e os comandos de simulação, verificando se todas as referências a estados, símbolos e autômatos são válidas:

- Estado inicial não declarado no conjunto de estados;
- Estado final não declarado no conjunto de estados;
- Estado de origem de transição não declarado;
- Estado de destino de transição não declarado;
- Símbolo da transição não pertence ao alfabeto;
- Comando de simulação referencia autômato inexistente.

7.4.3 Verificação de Determinismo

A terceira fase aplica-se exclusivamente a autômatos declarados como AFD e verifica duas condições:

- Ausência de transições ε (AFDs não podem ter transições vazias);
- Unicidade de transição: para cada par (estado, símbolo), deve existir no máximo uma transição definida.

Caso alguma dessas condições seja violada, o erro é reportado com indicação das transições conflitantes. Para AFNs, ambas as condições são permitidas, e nenhum erro é gerado.

Os erros semânticos são acumulados e apresentados em conjunto ao final da análise, permitindo que o estudante visualize todos os problemas de uma só vez.

7.5 SIMULADOR

O simulador é o componente responsável por executar a função de transição do autômato sobre uma cadeia de entrada, produzindo a sequência de configurações intermediárias percorridas durante o reconhecimento.

7.5.1 Estratégia de Exibição

Um autômato não determinístico pode encontrar-se em vários estados ao mesmo tempo, e a forma usual de simulá-lo consiste em propagar o conjunto de todos esses estados a cada passo, à maneira da construção de subconjuntos apresentada na Seção 2.5. Essa estratégia é fiel ao formalismo, mas produz uma exibição em que vários estados se acendem simultaneamente, sem indicar qual deles de fato conduz ao reconhecimento da cadeia.

O simulador da ferramenta adota, por isso, uma estratégia distinta: em vez de propagar a nuvem de estados, ele seleciona previamente um único caminho do autômato e o percorre passo a passo. A configuração exibida em cada instante contém, assim, um só estado, o que torna a execução legível como uma trajetória sobre o diagrama. Quando a cadeia pertence à linguagem, o caminho exibido é aceitador, isto é, parte do estado inicial, consome todos os símbolos da entrada e termina em um estado final. Quando não pertence, exibe-se o caminho que mais se aproxima da aceitação, segundo o critério descrito na subseção seguinte.

7.5.2 Seleção do Caminho

A escolha do caminho é feita por uma busca em largura sobre o espaço de configurações formado pelos pares (estado, quantidade de símbolos já consumidos). A partir do par inicial, a busca segue dois tipos de aresta: as transições ε, que mudam o estado sem avançar na entrada, e as transições rotuladas com o próximo símbolo da cadeia, que avançam uma posição. Como cada par é visitado uma única vez, os ciclos de transições ε não provocam repetição indefinida.

Entre todos os pares alcançados, elege-se o melhor por dois critérios, aplicados nesta ordem:

1. maior quantidade de símbolos consumidos da entrada;

2. menor distância até um estado final no grafo do autômato.

Esses dois critérios bastam para produzir os dois comportamentos desejados, sem que seja necessário distinguir previamente os casos de aceitação e de rejeição. Se a cadeia pertence à linguagem, existe um par que consumiu a entrada inteira e repousa sobre um estado final: o primeiro critério o coloca à frente de qualquer par que tenha consumido menos, e o segundo o coloca à frente dos demais pares que também consumiram tudo, por estar à distância zero de um estado final. O caminho reconstruído é, portanto, necessariamente aceitador. Se a cadeia não pertence, os mesmos critérios elegem o caminho que chega mais longe na entrada e, entre os que empatam nesse ponto, aquele que termina mais próximo de um estado final.

A distância mencionada no segundo critério é calculada uma única vez, antes da busca, por uma segunda busca em largura sobre o grafo invertido do autômato, partindo simultaneamente de todos os estados finais. Estados que não alcançam nenhum estado final ficam ausentes do resultado e são tratados como infinitamente distantes.

Por fim, o caminho é reconstruído a partir do par eleito, seguindo os apontadores de origem registrados durante a busca. Como a busca é feita em largura, o caminho obtido é também o mais curto entre os que levam àquele par.

7.5.3 Passo de Simulação

Determinado o caminho, a simulação consiste em percorrê-lo. Cada passo corresponde a um de dois tipos de transição:

1. Transição ε: o autômato muda de estado sem consumir símbolo da entrada;
2. Consumo de caractere: o autômato muda de estado e avança uma posição na cadeia.

A simulação termina quando o último passo do caminho é executado. A cadeia é aceita se o caminho consumiu a entrada por inteiro e terminou em um estado final; caso contrário, é rejeitada. Quando a rejeição decorre da ausência de transição disponível, e não do esgotamento da entrada, o simulador informa a posição do símbolo em que o caminho ficou preso, de modo que a interface possa destacá-lo.

O histórico completo de configurações é armazenado em um vetor, permitindo navegação tanto para frente quanto para trás durante a simulação visual.

7.6 INTERFACE GRÁFICA

A interface gráfica é implementada com a biblioteca Iced (Ramón, 2022), uma biblioteca *cross-platform* para Rust que segue a arquitetura Elm (*Model-View-Update*). A interface é composta por uma janela única dividida em três áreas principais.

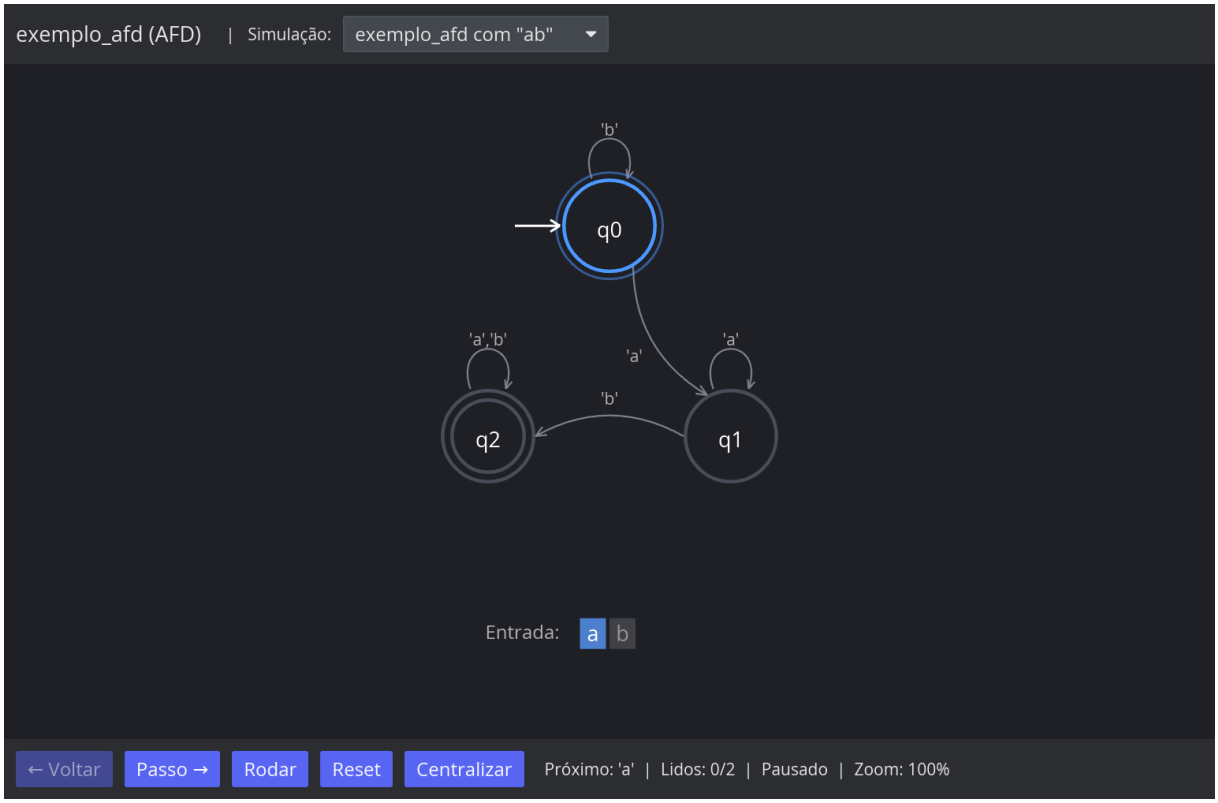
7.6.1 Layout da Interface

A janela da aplicação organiza-se verticalmente em:

- Cabeçalho: exibe o nome e o tipo do autômato selecionado, com um menu suspenso para escolher qual simulação executar;
- Canvas central: ocupa a maior parte da janela e renderiza o diagrama de estados do autômato, a cadeia de entrada com destaque para os caracteres já consumidos e o histórico das configurações percorridas;
- Barra de controle: oferece botões para avançar (`Passo`), retroceder (`Voltar`), reprodução automática (`Rodar/Pausar`), reinicialização (`Reset`) e recentralização da visão (`Centralizar`), além de exibir o progresso da simulação, o nível de ampliação corrente e o veredito final (aceita ou rejeita).

A Figura 13 apresenta a interface da ferramenta Autosim em execução, evidenciando as três áreas descritas: o cabeçalho, o canvas central com o diagrama renderizado a partir do autômato da Listagem 6.1 e a barra de controle inferior.

FIGURA 13 – VISÃO GERAL DA INTERFACE DA FERRAMENTA AUTOSIM



FONTE: Os autores (2026)

7.6.2 Renderização do Diagrama de Estados

O diagrama de estados é renderizado por meio da API *canvas* do Iced. O algoritmo de *layout* distribui os estados no espaço disponível de acordo com as seguintes regras:

- Para um único estado, o posicionamento é centralizado;
- Para dois estados, eles são dispostos horizontalmente com espaçamento proporcional ao raio;
- Para três ou mais estados, a disposição segue um arranjo circular com espaçamento angular uniforme.

Cada estado é desenhado como um círculo preenchido com o nome em texto branco. Estados de aceitação são representados com um círculo duplo (anel concêntrico). O estado inicial é indicado por uma seta apontando da esquerda. Durante a simulação, o estado corrente do caminho exibido é destacado com uma cor de realce

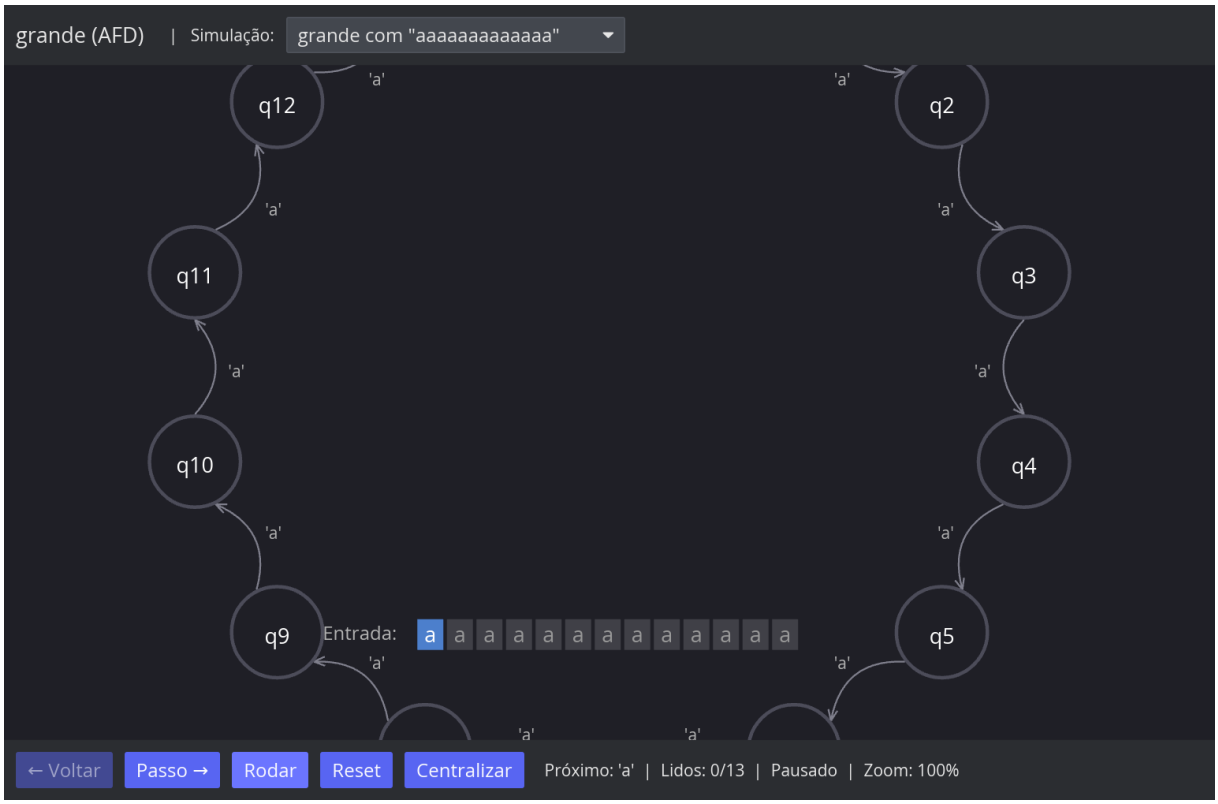
(azul durante a execução, verde para aceitação, vermelho para rejeição) e, enquanto a execução está em andamento, com um efeito pulsante.

As transições são renderizadas como arestas rotuladas entre os estados. Quando múltiplos símbolos compartilham a mesma origem e destino, os rótulos são concatenados. Transições ativas (executadas no passo corrente) são destacadas com espessura maior e cor brilhante.

Para acomodar autômatos com muitos estados, cujo arranjo pode exceder os limites da janela, o diagrama é desenhado sob uma câmera que oferece navegação livre por meio do *mouse*. O usuário desloca a visão (*pan*) arrastando com o botão esquerdo e ajusta a ampliação (*zoom*) com a roda de rolagem, que é ancorada na posição do cursor e limitada à faixa de 20% a 400%. O botão `Centralizar` restaura a visão original. A câmera é aplicada apenas ao diagrama de estados. A cadeia de entrada e o histórico de configurações permanecem fixos na tela, funcionando como um painel de informações sempre visível.

A Figura 14 ilustra esse recurso com um AFD de 14 estados: o arranjo circular excede a área visível da janela, tornando visível a necessidade da navegação por *pan* e *zoom* descrita acima.

FIGURA 14 – DIAGRAMA DE UM AFD DE 14 ESTADOS EXCEDENDO OS LIMITES DA JANELA



FONTE: Os autores (2026)

7.6.3 Simulação Visual Passo a Passo

A simulação visual sincroniza o estado do simulador com a renderização do canvas. A cada passo, o canvas é atualizado para refletir:

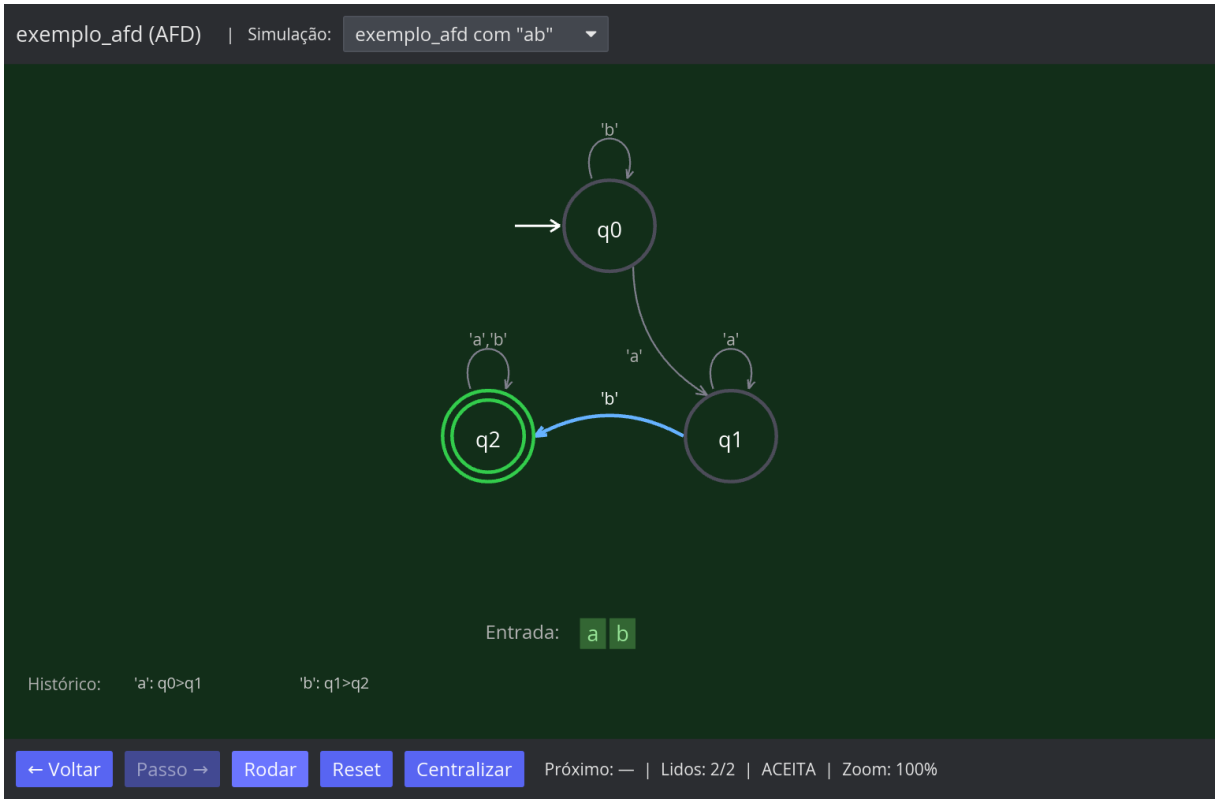
- O estado corrente do caminho (destacado visualmente);
- A transição executada no último passo (aresta destacada);
- A cadeia de entrada com caracteres já consumidos em verde, o caractere corrente em azul e os caracteres não lidos em cinza;
- O histórico de configurações, exibido como uma linha do tempo na parte inferior do canvas.

A reprodução automática avança a simulação em intervalos de 800 milissegundos, permitindo ao estudante acompanhar o processamento sem intervenção manual.

A Figura 15 mostra o resultado da reprodução automática sobre o AFD da Listagem 6.1 com a cadeia `"ab"`: o painel muda para a paleta verde de aceitação,

o estado `q2` é destacado como estado corrente e final, a última transição percorrida ($q_1 \xrightarrow{b} q_2$) é realçada em azul, a cadeia de entrada aparece integralmente consumida e o histórico de configurações (’a’: q0>q1, ’b’: q1>q2) é exibido abaixo do diagrama, junto do veredito `ACEITA` na barra de controle.

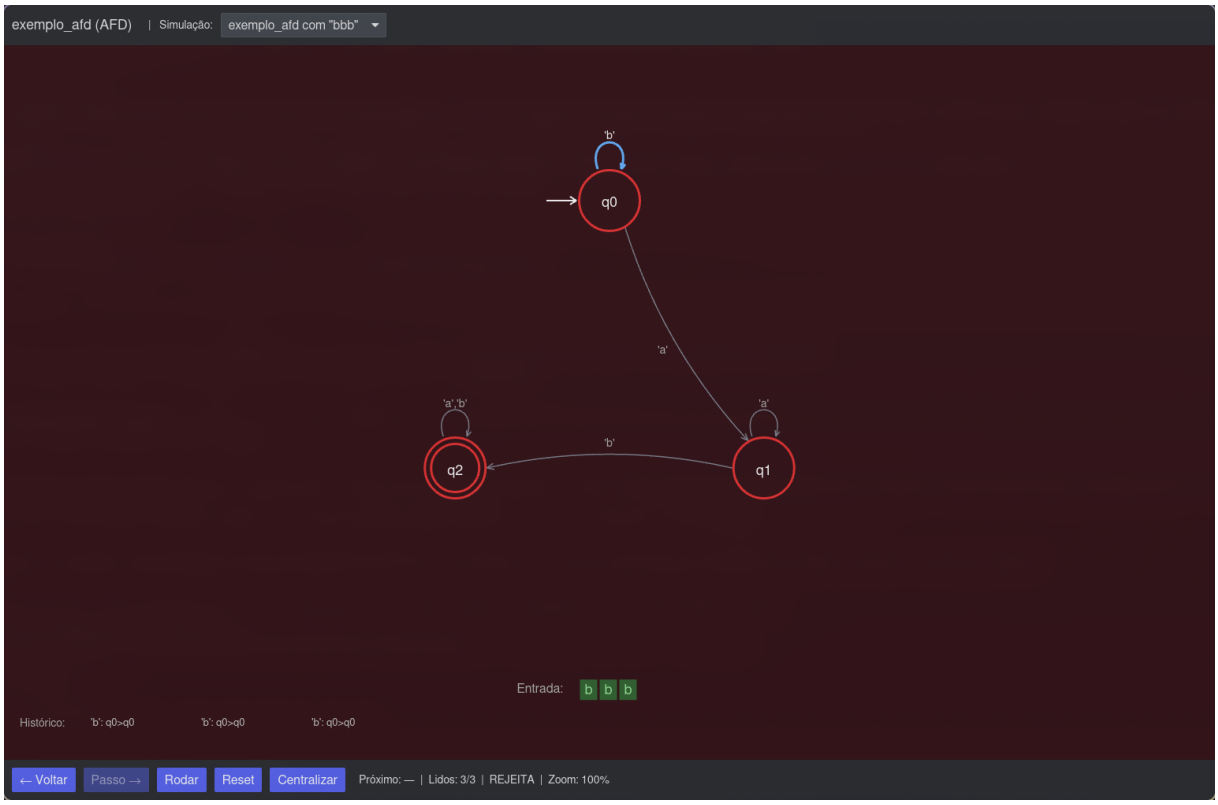
FIGURA 15 – SIMULAÇÃO CONCLUÍDA COM VEREDITO DE ACEITAÇÃO



FONTE: Os autores (2026)

De forma complementar, a Figura 16 apresenta o mesmo AFD processando a cadeia `"bbb"`, que não pertence à linguagem reconhecida: o painel adota a paleta vermelha de rejeição, o autômato permanece no estado `q0` (não final) após consumir integralmente a entrada por meio da transição de laço $q_0 \xrightarrow{b} q_0$, e a barra de controle exibe o veredito `REJEITA`. O contraste entre as duas paletas (verde para aceitação e vermelha para rejeição) reforça visualmente o desfecho da simulação.

FIGURA 16 – SIMULAÇÃO CONCLUÍDA COM VEREDITO DE REJEIÇÃO



FONTE: Os autores (2026)

7.7 DIAGNÓSTICO DE ERROS COM ARIADNE

Todas as fases do pipeline utilizam a biblioteca Ariadne para a geração de mensagens de erro. O diagnóstico inclui:

- O caminho do arquivo e a posição do erro (linha e coluna);
- Um trecho do código-fonte com realce da região problemática;
- Uma seta primária apontando para a localização exata do erro;
- Setas secundárias para informações complementares (como a declaração original de um estado duplicado);
- Mensagem descritiva em português.

Esse formato de diagnóstico, semelhante ao de compiladores modernos como o `rustc`, foi adotado por seu valor pedagógico: o estudante recebe informações claras e visualmente organizadas sobre cada erro, facilitando a correção do código-fonte.

8 RESULTADOS

Este capítulo apresenta os resultados obtidos com a implementação da ferramenta Autosim. A validação foi realizada por meio de uma suíte de testes automatizados, cobrindo as três fases de análise (léxica, sintática e semântica) e o simulador, totalizando 51 casos de teste. Além dos testes, foram conduzidas simulações com autômatos clássicos da literatura para verificar a correção do simulador e a qualidade dos diagnósticos de erro.

8.1 ESTRATÉGIA DE TESTES

Os testes foram organizados em quatro categorias: três delas correspondem às fases de análise do pipeline de compilação (léxica, sintática e semântica), e a quarta, ao simulador. Todos os testes são executados automaticamente pelo comando `cargo test` e seguem o padrão de testes unitários do ecossistema Rust (Klabnik; Nichols, 2019). O Quadro 8 apresenta um resumo quantitativo da cobertura de testes.

QUADRO 8 – RESUMO DOS TESTES AUTOMATIZADOS

Categoria	**Quantidade**	**Principais verificações**
Análise Léxica	12	Palavras reservadas, *case-insensitive*, literais, epsilon, comentários, erros
Análise Sintática	15	*Parsing* completo, AFN com epsilon, alfabeto vazio, múltiplos autômatos, erros
Análise Semântica	15	AFD válido, AFN válido, duplicatas, referências inválidas, determinismo
Simulação	9	Caminho aceitador, uso de epsilon, cadeia vazia, rejeição por falta de transição, navegação
Total	**51**	

FONTE: Os autores (2026)

8.2 TESTES DO ANALISADOR LÉXICO

Os 12 testes do analisador léxico verificam o reconhecimento correto de cada categoria de *token* e o tratamento de casos especiais:

- Palavras reservadas (1 teste): verifica que cada palavra-chave produz o *token* correspondente;

- *Case-insensitive* (1 teste): verifica que variações de caixa alta e baixa são igualmente reconhecidas (ex.: `ALFABETO`, `Automato`, `afd`);
- Aliases de epsilon (1 teste): verifica que `epsilon`, `eps`, `EPSILON` e `Eps` produzem o mesmo *token* `Epsilon`;
- Símbolos (1 teste): verifica o reconhecimento de `{`, `}`, `,` e `->`;
- Literais caractere e string (2 testes): verifica a correta extração do caractere interno em `'a'` e da string em `"ab"`;
- Identificadores (1 teste): verifica que `q0` e `exemplo_afd` são reconhecidos como `Ident`;
- Comentários (1 teste): verifica que comentários iniciados por `//` são ignorados;
- Espaços em branco (1 teste): verifica que espaços, tabulações e quebras de linha são ignorados;
- Programa completo (1 teste): verifica a sequência de *tokens* de um arquivo completo;
- Programa de AFN com epsilon (1 teste): verifica que a análise léxica de um arquivo descrevendo um AFN com transições ε produz os *tokens* `AFN` e `Epsilon`;
- *Token* inválido (1 teste): verifica que um caractere não reconhecido (como `%`) produz um erro com o *span* correto (9..10).

Todos os 12 testes do analisador léxico são aprovados com sucesso.

8.3 TESTES DO ANALISADOR SINTÁTICO

Os 15 testes do analisador sintático verificam a construção correta da AST a partir de entradas válidas e o tratamento adequado de entradas malformadas:

- Alfabeto apenas (1 teste): verifica que um programa contendo somente a declaração do alfabeto produz uma AST vazia para autômatos e simulações;
- Alfabeto vazio (1 teste): verifica que `alfabeto { }` é aceito;

- Vírgula final (1 teste): verifica que uma vírgula após o último elemento da lista é aceita;
- AFD completo (1 teste): verifica que todos os campos de um AFD (estados, inicial, finais, transições) são corretamente extraídos;
- AFN com epsilon (1 teste): verifica que transições ε são corretamente classificadas como `Symbol::Epsilon`;
- Transições vazias (1 teste): verifica que um bloco de transições vazio é aceito;
- Múltiplos autômatos (1 teste): verifica que dois autômatos no mesmo arquivo são ambos reconhecidos;
- Múltiplas simulações (1 teste): verifica que múltiplos comandos de simulação são registrados;
- *Case-insensitive* (1 teste): verifica que palavras em maiúsculas são aceitas pelo *parser*;
- Erro: palavra-chave ausente (1 teste): verifica que a ausência da palavra `inicial` gera mensagem de erro;
- Erro: símbolo inválido no alfabeto (1 teste): verifica que um identificador onde se esperava um caractere literal produz erro;
- Erro: tipo de autômato ausente (1 teste): verifica que a omissão de `AFD` ou `AFN` gera erro;
- Erro: bloco não terminado (1 teste): verifica que um bloco sem fechamento produz mensagem de erro;
- Erro: símbolo de transição inválido (1 teste): verifica que um identificador no lugar de um literal de caractere na transição gera erro;
- *Spans* como índices de *token* (1 teste): verifica que os *spans* armazenados na AST referem-se a índices de *token*, e não a deslocamentos de caractere, sendo a conversão realizada apenas na geração das mensagens de erro.

Todos os 15 testes do analisador sintático são aprovados.

8.4 TESTES DO ANALISADOR SEMÂNTICO

Os 15 testes do analisador semântico cobrem as verificações de consistência estrutural dos autômatos:

- AFD válido (1 teste): verifica que um AFD semanticamente correto produz uma tabela de símbolos completa;
- AFN com epsilon válido (1 teste): verifica que AFNs com transições ε são aceitos;
- Simulação válida (1 teste): verifica que uma referência a autômato existente no comando `simular` é aceita;
- Símbolo duplicado no alfabeto (1 teste): verifica que a repetição de um símbolo no alfabeto é detectada;
- Estado duplicado (1 teste): verifica que a declaração do mesmo estado duas vezes é reportada;
- Autômato duplicado (1 teste): verifica que dois autômatos com o mesmo nome são detectados;
- Estado inicial não declarado (1 teste): verifica que uma referência a estado inexistente como inicial é reportada;
- Estado final não declarado (1 teste): similar ao anterior para estados finais;
- Estado de transição não declarado (1 teste): verifica a detecção de estados não declarados usados em transições;
- Símbolo fora do alfabeto (1 teste): verifica que um símbolo não pertencente ao alfabeto é rejeitado;
- Simulação de autômato inexistente (1 teste): verifica que referência a autômato não declarado é detectada;
- AFD não-determinístico (1 teste): verifica que transições conflitantes em AFD são reportadas;
- AFD com transição epsilon (1 teste): verifica que transições ε em AFD são rejeitadas;

- AFN permite não-determinismo (1 teste): verifica que transições conflitantes em AFN são permitidas;
- Acumulação de múltiplos erros (1 teste): verifica que o analisador não interrompe a análise no primeiro erro, acumulando ao menos 3 erros simultâneos.

Todos os 15 testes do analisador semântico são aprovados. O teste de acumulação de erros confirma que o analisador semântico é capaz de detectar e reportar, em uma única execução, problemas de diferentes naturezas (estado inicial ausente, estado final ausente, símbolo fora do alfabeto), evitando que o estudante precise corrigir um erro de cada vez.

8.5 TESTES DO SIMULADOR

Os 9 testes do simulador verificam a estratégia de caminho único descrita na Seção 7.5. Os casos utilizam os próprios arquivos de exemplo distribuídos com a ferramenta, de modo que esses exemplos funcionem também como massa de teste. Em vários deles, o caminho produzido é submetido a uma verificação de boa formação, que confirma que ele parte do estado inicial, que cada passo corresponde a uma transição existente no autômato e que o veredito decorre do próprio caminho, isto é, que a cadeia é aceita se, e somente se, o caminho consome toda a entrada e termina em um estado final. Os testes são:

- Caminho aceitador (1 teste): verifica que, para uma cadeia aceita por um AFN com múltiplos caminhos possíveis, o caminho exibido consome toda a entrada e termina em um estado final;
- Um único estado por configuração (1 teste): verifica que cada configuração do histórico contém exatamente um estado, e não o conjunto de estados simultâneos;
- Uso de transição ε (1 teste): verifica que o caminho percorre uma transição ε quando ela é necessária para a aceitação, como em `aabb` no AFN-ε da Listagem 6.2;
- Cadeia vazia (1 teste): verifica que a cadeia vazia é aceita, sem nenhum passo, quando o estado inicial é final;

- Rejeição por falta de transição (1 teste): verifica que, para a cadeia `ba`, o caminho avança o máximo possível e o simulador informa a posição do símbolo em que ficou preso;
- Rejeição com a entrada consumida (1 teste): verifica que, quando a entrada é lida por inteiro sem atingir um estado final, o caminho termina no estado mais próximo da aceitação;
- Navegação passo a passo (1 teste): verifica que avançar, retroceder e reiniciar a simulação percorrem e desfazem o mesmo caminho;
- Exemplos válidos (1 teste): verifica que todas as simulações declaradas nos arquivos de exemplo sem erros propositais produzem caminhos bem formados;
- Vereditos dos exemplos (1 teste): verifica que os vereditos de aceitação e rejeição conferem com a linguagem descrita por cada autômato de exemplo.

Todos os 9 testes do simulador são aprovados. Em conjunto, eles confirmam que a escolha de um único caminho preserva a correção do reconhecimento: o caminho exibido é aceitador sempre que a cadeia pertence à linguagem, e o veredito obtido coincide com o esperado.

8.6 SIMULAÇÕES COM AUTÔMATOS CLÁSSICOS

Para validar a correção do simulador, foram executadas simulações com autômatos clássicos da literatura, descritos em Hopcroft, Motwani e Ullman (2006) e Sipser (2012). O Quadro 9 apresenta os resultados obtidos.

Todas as simulações produziram o veredito esperado, confirmando a correção do simulador tanto para AFDs quanto para AFNs com transições ε.

8.7 CENÁRIOS DE DIAGNÓSTICO DE ERRO

A qualidade dos diagnósticos de erro foi avaliada por meio de arquivos de teste com erros propositais, simulando situações reais de uso por estudantes. O Quadro 10 apresenta os cenários testados e os resultados obtidos.

QUADRO 9 – SIMULAÇÕES REALIZADAS COM AUTÔMATOS CLÁSSICOS

Autômato	**Cadeia**	**Esperado**	**Obtido**
AFD: cadeias contendo 01	`1101`	Aceita	Aceita
AFD: cadeias contendo 01	`111`	Rejeita	Rejeita
AFD: cadeias contendo 01	`101`	Aceita	Aceita
AFD: cadeias contendo 01	`000`	Rejeita	Rejeita
AFN: cadeias terminadas em 01	`001`	Aceita	Aceita
AFN: cadeias terminadas em 01	`010`	Rejeita	Rejeita
AFN-ε: a^*b^*	`aaa`	Aceita	Aceita
AFN-ε: a^*b^*	`aabb`	Aceita	Aceita
AFN-ε: a^*b^*	`ba`	Rejeita	Rejeita
AFN-ε: a^*b^*	ε	Aceita	Aceita

FONTE: Os autores (2026)

QUADRO 10 – CENÁRIOS DE TESTE DE DIAGNÓSTICO DE ERRO

Arquivo de teste	**Tipo de erro**	**Diagnóstico gerado**
`sema_error.asl`	4 erros semânticos	4 mensagens localizadas
`sema_dfa_epsilon.asl`	AFD com ε	AFD não pode ter epsilon
`sema_nondeterministic.asl`	AFD não-determinístico	Transição conflitante
`parse_error.asl`	Falta `inicial`	"esperava ’inicial’"
`symbol_error.asl`	Caractere inválido	"token inválido"
`error_line3.asl`	Caractere inválido	"token inválido ’@’"(linha 3)

FONTE: Os autores (2026)

Em todos os cenários, as mensagens de erro incluíram o número da linha e coluna, o trecho do código-fonte com realce e uma descrição do problema em português, confirmando a eficácia da estratégia de diagnóstico adotada.

9 CONCLUSÃO

Este trabalho apresentou o desenvolvimento de uma ferramenta educacional baseada em Linguagem de Domínio Específico (DSL) para o apoio ao ensino de autômatos finitos com simulação visual interativa. A ferramenta, denominada Autosim, foi projetada para preencher a lacuna identificada nas ferramentas existentes entre a descrição formal textual e a visualização gráfica, oferecendo uma abordagem que integra o rigor da compilação com a interatividade da simulação. Nesse processo, foi projetada e implementada uma DSL textual com sintaxe declarativa em português para a descrição formal de autômatos finitos determinísticos e não determinísticos com transições ε, cuja gramática foi formalizada em notação EBNF. Sobre essa linguagem, foi implementado um compilador pedagógico completo, reunindo análise léxica, realizada com a biblioteca Logos, análise sintática, com base na biblioteca Chumsky, e análise semântica, capaz de detectar erros estruturais e conceituais com mensagens de diagnóstico localizadas e coloridas. Complementarmente, desenvolveu-se um simulador que processa cadeias de entrada sobre os autômatos descritos na DSL, exibindo o caminho aceitador quando a cadeia é reconhecida e, quando não é, aquele que mais se aproxima da aceitação, com histórico completo das configurações intermediárias e navegação bidirecional durante a execução. Por fim, construiu-se uma interface gráfica interativa que renderiza automaticamente o diagrama de estados a partir do texto fonte e anima a simulação passo a passo, com destaque visual para o estado corrente e as transições percorridas. A validação da ferramenta foi realizada por meio de 51 testes automatizados, distribuídos entre as fases léxica, sintática e semântica e o simulador, além de simulações com autômatos clássicos da literatura, com todos os resultados concordando com o esperado, o que confirma a correção do simulador e a qualidade dos diagnósticos gerados.

No entanto, apesar desses resultados positivos, a ferramenta apresenta limitações que devem ser reconhecidas. Todos os autômatos de um mesmo arquivo compartilham um único alfabeto, o que pode ser restritivo para cenários que exijam alfabetos distintos. Não há suporte a um modo de execução puramente textual em linha de comando, sem a interface gráfica, e cada execução processa apenas um arquivo fonte, sem suporte a módulos ou inclusão de múltiplos arquivos. A interface gráfica

também não inclui um editor de texto embutido com realce de sintaxe ou completamento automático, e a ferramenta ainda não oferece operações como conversão automática de autômatos não determinísticos para determinísticos, minimização de estados ou extração de gramáticas formais.

Diante desse cenário, os desdobramentos futuros buscam superar tais limitações e ampliar o alcance pedagógico da ferramenta. Pretende-se integrar um editor de texto com realce de sintaxe diretamente na interface gráfica, estendendo-o a permitir edição e visualização simultâneas, além de suportar alfabetos distintos por autômato. Também se planeja implementar algoritmos de conversão de autômatos não determinísticos para determinísticos pela construção de subconjuntos, de minimização de estados e de conversão entre autômatos e expressões regulares, bem como adicionar um modo de execução textual em linha de comando, útil para automação e testes. Em uma perspectiva mais ampla, a ferramenta pode avançar pelos níveis superiores da Hierarquia de Chomsky, apresentada no Quadro 1, incorporando à linguagem a descrição e a simulação de autômatos com pilha, reconhecedores das linguagens livres de contexto, e de máquinas de Turing, cuja execução passo a passo, com a visualização da fita e do cabeçote, contribuiria para o ensino de tópicos mais avançados da Teoria da Computação, como decidibilidade e computabilidade (Sipser, 2012; Hopcroft; Motwani; Ullman, 2006). Outras direções incluem estender a ferramenta à descrição e simulação de gramáticas formais, permitir a exportação do diagrama de estados como imagem para inclusão em relatórios e realizar estudos controlados com estudantes, a fim de avaliar de modo rigoroso o impacto pedagógico da ferramenta em comparação com métodos tradicionais de ensino.

REFERÊNCIAS

AHO, A. V. *et al.* **Compiladores: Princípios, Técnicas e Ferramentas**. 2. ed. São Paulo: Pearson Addison-Wesley, 2008.

BARRETTO, J. **chumsky: A parser library for humans with powerful error recovery**. 2022. Disponível em: https://github.com/zesterer/chumsky. Acesso em: 29 abr. 2026.

BRUZZONE, F. **Exploiting Finite State Automata for Efficient Lexical Analysis: A Rust Implementation**. 2023. Disponível em: https://federicobruzzone.github.io/posts/exploiting-finite-state-automata-for-efficient-lexical-analysis-a-rust-implementation.html. Acesso em: 9 ago. 2026.

COUPRIE, G. **nom: Rust parser combinators framework**. 2015. Disponível em: https://github.com/rust-bakery/nom. Acesso em: 29 abr. 2026.

DIVERIO, T. A.; MENEZES, P. B. **Teoria da Computação: Máquinas Universais e Computabilidade**. 3. ed. Porto Alegre: Bookman, 2011.

FOWLER, M.; PARSONS, R. **Domain-Specific Languages**. Upper Saddle River, NJ: Addison-Wesley Professional, 2010. (Addison-Wesley Signature Series).

HIRSZ, M. **logos: Create ridiculously fast Lexers**. 2021. V0.16. Disponível em: https://github.com/maciejhirsz/logos. Acesso em: 15 jun. 2026.

HOPCROFT, J. E.; MOTWANI, R.; ULLMAN, J. D. **Introduction to Automata Theory, Languages, and Computation**. 3. ed. Boston, MA: Pearson Addison-Wesley, 2006.

JUKEMURA, A. L.; NASCIMENTO, H. A. D.; UCHÔA, J. Q. GAM: Um simulador para auxiliar o ensino de linguagens formais e de autômatos. **Anais do XIII Workshop sobre Educação em Computação – XXV Congresso da Sociedade Brasileira de Computação**, São Leopoldo, 2005.

KIENETZ, E. B.; CANAL, A. P. Simulador de autômatos finitos determinísticos: AutomatoGraph. **Disciplinarum Scientia. Série: Ciências Naturais e Tecnológicas**, v. 5, n. 1, p. 1–9, 2016.

KLABNIK, S.; NICHOLS, C. **The Rust Programming Language**. San Francisco: No Starch Press, 2019.

LEWIS, H. R.; PAPADIMITRIOU, C. H. **Elements of the Theory of Computation**. 2. ed. Upper Saddle River, NJ: Prentice Hall, 1998.

MORAZÁN, M. T.; ANTUNEZ, R. Functional automata: Formal languages for computer science students. In: **Proceedings of the 3rd International Workshop on Trends in Functional Programming in Education (TFPIE 2014)**. [S.l.]: Open Publishing Association, 2014. (Electronic Proceedings in Theoretical Computer Science, v. 170), p. 19–32.

NUNES, G. C. **Simulador de autômatos e máquinas de Turing**. Monografia (Trabalho de Conclusão de Curso), Florianópolis, 2017.

RAMÓN, H. **iced: A cross-platform GUI library for Rust**. 2022. V0.14. Disponível em: https://github.com/iced-rs/iced. Acesso em: 15 jun. 2026.

RODGER, S. H.; FINLEY, T. W. **JFLAP: An Interactive Formal Languages and Automata Package**. Sudbury, MA: Jones and Bartlett Publishers, 2006.

SIPSER, M. **Introduction to the Theory of Computation**. 3. ed. Boston, MA: Cengage Learning, 2012.

TISELICE, D. **pest: The Elegant Parser**. 2018. Disponível em: https://pest.rs. Acesso em: 29 abr. 2026.

VIEIRA, J. T.; BECKMANN, K. S. **Autosim: Linguagem de Domínio Específico para Simulação de Autômatos Finitos**. 2026. Disponível em: https://github.com/KsBeckmann/autosim. Acesso em: 9 ago. 2026.

VIEIRA, N. J. **Linguagens e Máquinas: Uma Introdução aos Fundamentos da Computação**. Belo Horizonte: Departamento de Ciência da Computação, Universidade Federal de Minas Gerais, 2004. Notas de aula.